

East Central University

UNDERGRADUATE BIOINFORMATICS GUIDEBOOK

NGS RNAseq

Caden Anthony Elizalde, B.S, B.A.

Master's Thesis
Biological Sciences - Health Sciences
Spring 2026



ECU

Table of Contents

Introduction.....	7
--------------------------	----------

Chapter 1: The Basics.....	9
-----------------------------------	----------

What is Bioinformatics?.....	9
What Skill Sets Should a Bioinformatician Have?.....	10
Setting Up Your Computer.....	12
Setting up Linux.....	14
BASH Crash Course.....	20
Accessing and Navigating the Terminal.....	22
Essential Commands for the Command Line.....	29
Setting Up Essential Software.....	38

Chapter 2 - Example Script.....	41
--	-----------

Outlining a plan.....	42
RNAseq Basics.....	42
Rough Plan.....	44
Preparation.....	45
Import Libraries - Gather Your Tools.....	48
Creating Helper Functions.....	50
Main Function - Why is it There?.....	59
Creating Arguments.....	61
Pre-Flight Check.....	64
Verify Fastq Files.....	65
Setup Output Directories.....	67
Download Genome Data.....	69
Locate FASTA and GTF.....	72
Build HISAT2 Index.....	74
Run FastQC.....	76
HISAT2 Alignment Loop.....	77
FeatureCounts.....	80

Quality Control.....	81
R Analysis - What do the numbers mean?.....	82
Summary.....	93
 Chapter 3 - Analysis.....	 94
Quality Control Check.....	95
MultiQC.....	95
Principal Component Analysis (PCA).....	109
Dispersion Plot.....	112
Analysis and Graphing.....	115
Differentially Expressed Genes Table.....	115
Volcano Plot - Visualize Significant Genes.....	117
GSEA Plot - Contextualize in Biological Pathways.....	121
 Chapter 4 - Conclusions.....	 123
 Appendices.....	 127
Appendix A - Important Commands.....	127
Appendix B - BASH Aliasises & Shortcuts.....	129
Appendix C - Useful Bioinformatics Software.....	131
 Sources.....	 132

Introduction

If you are reading this then you are likely the unfortunate soul that was assigned to be the resident undergraduate bioinformaticist for your specific research project. If you are a student reading this voluntarily, I seriously question your sanity. However, know that you are in good company with the author of this thesis and the scientific community as a whole.

No matter how you ended up here, the goal of this thesis is to give a brief introduction to bioinformatics, walk through a common example bioinformatics pipeline, and explain the pros and cons between two common ways of running a bioinformatics pipeline. I have chosen RNAseq differential expression as the example pipeline for this thesis as it is relatively simple to explain and acts as a good baseline for more complex bioinformatics pipelines such as methylseq and ChIPseq. As of the time of writing, RNAseq is also an extremely in demand skill that is capable of producing tangible results.

To prevent this thesis from recapping the entire history of biology, experimental design, and computer science this thesis is written with certain assumptions about the reader. While there is no expectation for the reader to have a computer science background, basic competency with computers is assumed. Additionally, there will be some higher level biology concepts appropriate for an undergraduate. If you experience any confusion while reading this, I highly recommend talking with some of the faculty members in the biology department to get a basic familiarity with the concept.

Additionally, a glossary is provided in the back of the thesis that has simple definitions of many of the terms used in this document.

The structure of the thesis itself will be split into three parts. Section one will consist of an introduction of what bioinformatics is and some information on how to effectively navigate your computer terminal. The second section is a detailed explanation of a pipeline I created to analyze a sample RNAseq dataset. The final section explains exactly how to analyze the results of the pipeline complete with both good and bad examples.

By the end of this thesis, you should have the skills to effectively run RNAseq pipelines and understand what the results mean. While we are focusing RNAseq, many of the skills learned in this project will apply to many other bioinformatics projects. Therefore, if RNAseq is not the protocol you need, this thesis should still be beneficial (especially if you have a foundation in biology but not computer science).

Thank you for taking the time to read this and let's get started with a brief crash course on bioinformatics.

Chapter 1: The Basics

What is Bioinformatics?

Bioinformatics can be seen as the intersection of Biology, Statistics, and Computer Science. Specifically, it is the process of taking large-scale data gained from some kind of experiment and turning it into meaningful results using computational models. The primary goal of bioinformatics is to translate the complexities of biology into tangible and workable data, then turn that data into insights into biology as a whole.^{1,2}

Explained another way, bioinformatics is the process of turning biological measurements into meaningful insights.

While bioinformatics is a bit more complex than turning A, G, T's, and C's into statistics, it isn't that much more complex at its core. In fact, most bioinformatics projects fall into one of four categories: Assembly, Resequencing, Classification, and Quantification. These categories focus on identifying biological sequences, differences, makeup, and outcomes respectively. Further explanation for each of the four categories can be seen below in **Table 1**.

Table 1.1: Bioinformatics Project Types

Project Category	Description	Examples
Assembly	Building the nucleotide sequence of an organism.	Human Genome Project
Resequencing	Identifying genomic variants or mutations in an organism's genome.	Identifying genetic variants between German Shepherds and Bulldogs in the family <i>Canis lupus familiaris</i> , etc.
Classification	Determining the species makeup of a larger population of organisms.	Metagenomic classification of the human gut microbiota, etc.
Quantification	Using DNA sequences to determine the biological characteristics of a cell or organism.	RNAseq, MeSeq, etc.

Table 1.1. A table explaining the 4 different categories of bioinformatics as well as some notable examples of each.

There is a good chance that, since the time of writing, new techniques or even categories may need to be considered. As new biological techniques are created and our fundamental understanding of biology evolves, bioinformatics changes as well. For example, bioinformatics was founded on genomics in an attempt to make sense of The Human Genome Project. As our understanding of biology advanced, transcriptomics (RNA), proteomics (proteins), epigenomics (non-coding DNA), and other -omics developed. While it may seem like training to be a bioinformatician is aiming for a moving target, the fundamentals are likely to remain consistent. These fundamentals are what I hope to teach you.

What Skill Sets Should a Bioinformatician Have?

Asking “What skills should a bioinformatician have?” is much like asking “What skills should a biologist have?” In both scenarios, the majority of skills you will need to do proper analysis are project-specific²¹. However, much in the same way that there are foundational skills that nearly every biologist will learn (pipetting, aseptic technique, etc.), there are also fundamental techniques that need to be learned for bioinformatics. The most important of which is being able to comfortably navigate your computer’s “command line interface” or CLI.

Most of our interactions with computers are done through a graphical user interface (or GUI) which consists of your desktop, mouse, web browsers, and everything else that comes to mind when you think of a modern computer. This makes accessing computers extremely user-friendly by combining complex processes into simple mouse clicks and putting up guard rails that make it hard to break your computer. While this is nice for everyday use, the tradeoff is that it limits what you can do to what the developers thought you would need. As bioinformaticians, we need to talk directly to the computer through the command line to ensure that it is doing *exactly* what we tell it to do.

Unfortunately, this means that an introductory knowledge of programming is needed to be successful in bioinformatics. While this skill is relatively trivial for people with a computer science background, it is the biggest hurdle that biologists will have in their transition to bioinformatics. While it will likely be frustrating at first, it will get easier

in time. I should note that there are non-command line ‘alternatives’ like Galaxy, I highly recommend avoiding these. These programs are easy to use, but are difficult to customize and troubleshoot. In addition, most open source and public bioinformatic tools rely on the command line, limiting yourself to programs like Galaxy isolates you from a community and tools that could be incredibly useful.

Setting Up Your Computer

As we mentioned in the previous section, the most important thing we need access to is something called a “Unix-based operating system” or Unix. Unix is an operating system, like Microsoft's Windows, that works directly from the command line. However, Unix is stripped of most of the bells and whistles that most modern operating systems have, making it extremely fast and efficient.

As a side note, “UNIX” is technically a proprietary product created by Ken Thompson, Dennis Ritchie, and their team at AT&T’s Bell Lab.²³ However, the open source community has functionally recreated the UNIX operating system and released to the public what we know as Linux. Today, Linux that has become the standard for most devices that just want to run simple commands. There are slightly different versions of Linux that are managed and released to the public in packages called distributions – or distros for short. At the current moment there are seven main distros with countless other community made distros that fulfill a specific purpose. See [Figure 1](#) to see a visual representation of this hierarchy of distros.

Figure 1.1: Linux Distribution Hierarchy

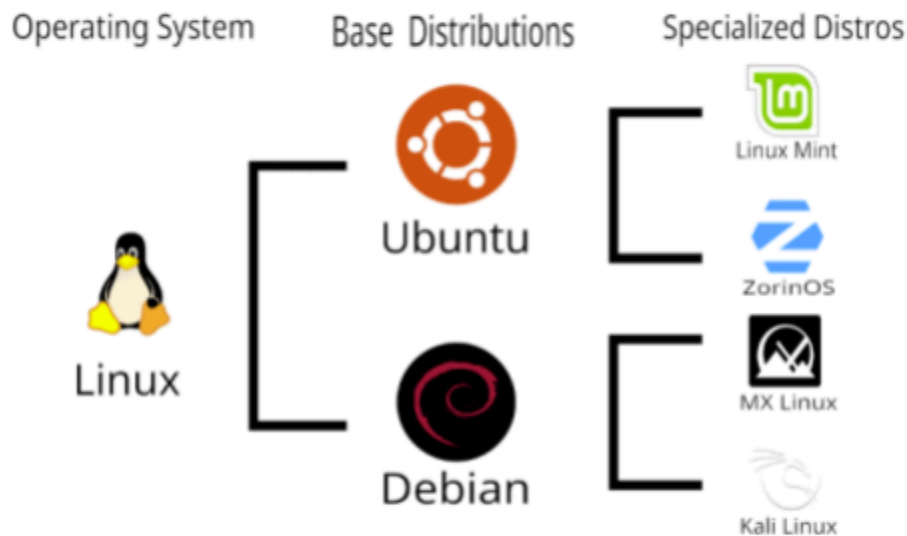


Figure 1.1. A graphical representation to explain linux terminology and how distributions are structured.

For this thesis, we will be using a computer running an Ubuntu-based Linux distribution. Many, if not all, of the bioinformatic software packages and tools can be run on macOS and Windows; however, I suggest Linux for two reasons: First, Linux is highly customizable and open source, while macOS and Windows are proprietary and built for a general audience. This means that if you need to go into the internals for whatever reason, fewer guardrails could get in the way of troubleshooting (note that these guardrails exist for a reason, so more care is needed when making changes like these). The second reason is simply that I am more familiar with Linux as it is what I have run during my time as a master's student. Therefore, I am able to talk in more depth about and anticipate troubleshooting for Linux better than I can for other operating

systems. However, as long as you have access to a Unix based command line, most of the information here will be directly transferable, no matter the OS you are using.

Setting up Linux²⁵

The work on this thesis will be done using the Linux Mint. I find this distribution to be the best for newer bioinformaticists since it comes with a GUI that looks and functions similar to Windows and is user-friendly straight out of the box. The following section will be a specific explanation on setting up Linux Mint; however, any Ubuntu-based Linux distro will have nearly identical results when commands are entered into the command line interface (or CLI). Non-Ubuntu distros (including Unix-based macOS) will be similar, but diverge in key areas. Windows will diverge even further.

This thesis will give a broad explanation of the steps to install Linux Mint as of the time of writing. However, small details may change slightly over time or differ based on hardware. Please use this as a supplement for the official installation documentation that can be found on the website of the distro of choice. Currently, the Linux Mint installation guide can be found at

<https://linuxmint-installation-guide.readthedocs.io/en/latest/>. **Additionally, if you or a mentor already have a bootable media, please skip to the section: [Boot and Install Linux](#).**

Creating a Bootable Media

The easiest way to install an OS on your computer is through what is called a boot stick. This is usually a USB stick that contains what is called an ISO image. You can think of an ISO file as an Ikea box for a new table you hired your computer to build; It contains all of the parts needed to build the OS as well as instructions for your computer to do it²⁴ resulting in the computer building an *exact* replica of what is on the ISO file. Most Linux Operating Systems, especially free ones, are created and maintained by an asynchronous group of individuals instead of large companies. Most of these individuals won't have the funds to set up a server that will allow you to download the ISO image at a reasonable speed. These groups will go to organizations or individuals that are willing to "Mirror" their site. That is where we are going to get our ISO image.

When you navigate to Linux Mint's download mirror site, you should be met with a list of organizations from all around the world. Each of these sites is hosting copies of the official downloads provided by Linux Mint. Click on the location that is closest to you for the fastest download ("Purdue Linux Users Group" is currently the closest mirror to ECU). This will take you to a stripped down page with only a title and links. While this page will likely raise some red for someone seeing a page like this for the first time, the simplicity is purposeful; making a site look pretty takes time, resources, and requires more maintenance. Most mirrors will look similar, especially if they are being hosted voluntarily. To double-check the site's safety, click on the search bar and look for "https://" at the beginning of the URL (this signifies that the website is encrypted and

malicious actors can't eavesdrop on you. Click on the "stable/" directory (comp-sci term for folder) to navigate to another page that will list various Linux Mint versions that have been bug-tested and are approved for release. Click on the latest release and you should see a list of Linux Mint files that end in '.iso' as well as two additional files that will have 'sha256sum' in their names. The different ISO images are for different subtypes of Linux Mint that have differing amounts of features. I recommend downloading the most fully loaded version into an empty folder. At the moment, this version is "Cinnamon," but the different versions and their capabilities can be double-checked on the official website. In addition, download the two 'sha256sum' files into that same folder by right-clicking on the link, then clicking "Save as" (**Note: do not copy paste the contents into a different text file. The verification process counts this as tampering and will cause the authentication check to flag the false positive**). I will explain what these files do next.

If you are copying an OS image to your computer, it is extremely important to know you can trust the source; building a replica of something on your computer is extremely convenient and powerful, but that means that it is also extremely dangerous if used for malicious purposes. In order to ensure that the Linux Mint file that you download is an unmodified official image, the OS you are downloading will provide a checksum file. This file is essentially a unique fingerprint that can be checked against the official download to ensure that the ISO you download is what you think it is.

At this point, you should have an ISO image and two sha256 checksum files in a folder. The next step is to double-check those files using the checksums to ensure that the files have not been tampered with. While it is rare for people to tamper with files on an official mirror, occasionally files can get corrupted during the download process which, for the code that will run your computer, can be just as bad as if it was tampered with. Also, it is good general practice to check any image file downloaded over the internet whenever you can. We do this by verifying 2 things: integrity (has it been modified?) and authenticity (is it official?).

In order to verify the ISO image, we are going to use the command line for the first time. On macOS, first open the “Terminal” app installed on your Mac. Then enter the command `shasum -a 256 yourfile.iso`. Essentially, this will run the file through a complex math equation that will create a unique line of characters that is unique to the file. Any modifications made to the file (even renaming it) will cause the character string to change. This string should be identical to the string located in the “sha256.txt” file. If it doesn’t, there are modifications in the ISO image that could indicate that it’s been tampered with or corrupted; delete the download and try again. If they match, you know that the file is what it says it is.

We know that the file is a one-to-one copy, but how do we know this is an official file and not a counterfeit? In order to find this out, we need to do an authenticity check using a command-line program called gpg to check the sha256sum with the one that is

verified by Linux Mint. Once again, open up the command line the same way you did in the last step and copy this into the command line:

```
gpg --keyserver hkp://keys.openpgp.org:80 --recv-key  
27DEB15644C6B3CF3BD7D291300F846BA25BAE09
```

This pulls the official key uploaded by Linux Mint to your computer. The `recv-key` option comes straight from Linux Mint's website, and we can verify that it came from Linux Mint with the command:

```
gpg --list-key --with-fingerprint A25BAE09
```

This should produce the output below:

```
pub      rsa4096 2016-06-07 [SC]  
          27DE B156 44C6 B3CF 3BD7  D291 300F 846B A25B AE09  
uid           [ unknown] Linux Mint ISO Signing Key  
<root@linuxmint.com>
```

Notice how the output confirms that it is a “Linux Mint ISO Signing Key” This confirms that the imported key comes from Linux Mint, and we can compare it to our ISO sha256 key by entering the command `gpg --verify sha256sum.txt.gpg sha256sum.txt`. If the output tells you the file signature is good, then you know that the fingerprint in the “sha256sum.txt” and therefore in your ISO file is legit, and you are ready to proceed!

Now that we have a legit ISO file, we need to write it to a bootable medium like a USB stick that will tell the computer how to install the OS onto your computer. Using the analogy of the Ikea table before, the file we just got is the equivalent of packing up the physical lumber and tools into the box; writing that file to the USB stick is the equivalent

of adding the manual. To do this, we need an image writer software that is capable of doing this. Both Linux Mint and I recommend [Etcher²⁶](#) as it is both user-friendly and reliable. Follow the instructions of the image writer you use to write the ISO file to the chosen USB stick **(Important Note: Make sure to remove any important data from the stick BEFORE writing the image. The formatting process will delete all previous data on the USB and it CANNOT be recovered.**

Congratulations! You are now the proud owner of an OS on a stick! Now comes the fun part; let's install your handywork.

Boot and Install Linux.

Whether created or inherited, you should be in possession of a bootable media, most likely a USB stick, containing a Linux Mint ISO image. Now, let's install it on a computer. Luckily, this step is substantially easier than the previous step.

First, we need to navigate to your computer's Basic Input/Output System (BIOS) menu. Your computer's BIOS is firmware that checks to see that your computer is good to go before loading your OS, and can be accessed by pressing a special key as your computer turns on. Unfortunately, the specific button differs from computer to computer; however, the specific button is often displayed on the monitor as the computer turns on (Usual culprits include Escape, F1, F2, F8, F10, F11, F12, and Delete). On Mac, the BIOS can be accessed by holding either the "Alt." or Options key while your system boots up. Insert the USB into your computer and navigate to the system BIOS. Once

you enter the system BIOS, there should be a section or tab named “Boot Order.” Once you get to that section, there should be an entry named either “Linux Mint” or “USB Boot.” Move this boot option to the top of the list and exit the system BIOS. Save changes and exit the BIOS menu. If everything goes well, the next time you boot up your computer, it should start Linux Mint.

If you see the Linux Mint homepage, you are on the right track. However, we are not entirely done yet. What we are looking at is something called a “Live Boot Session.” This is a version of your OS that exists directly on the USB and not your computer. If we make any changes to this version, they will be erased when we boot it up again. In order to make changes permanent, we need to install a version of this boot directly on your computer’s storage. To do this, click the “Install Linux Mint” icon at the top left of the desktop and follow the steps.

BASH Crash Course

Like I said earlier, one skill that any bioinformatician needs is a foundational understanding of the command line. As a note, terms like “the command line”, “the command line interface (CLI),” “the terminal,” “Unix,” and oftentimes even “BASH” are used interchangeably in the field and throughout this thesis. While there are technically slight differences between these terms, all of these terms are colloquially used to describe the computer terminal.

If you haven't coded before, your first exposure will likely be extremely overwhelming. Additionally, as you go through examples, you will likely get many, *many* errors that may be discouraging as you go through the process of learning bioinformatics. This is completely normal and a rite of passage for anyone starting to code. Unfortunately, these errors will continue for as long as you code. What separates, new coders from career coders is how fast they deal with those error codes. You will see yourself get better at that with time.

Additionally, seeing that many error messages and making that many mistakes may tempt you to subvert the command line entirely using a GUI tool like Galaxy or have AI code for you. While I would highly discourage using a GUI tool for reasons previously described, AI can be an incredibly valuable tool for both coding and learning.

To be clear, I **don't** recommend having AI code everything for you. While this may seem tempting, and AI may be able to produce something that works, AI is currently bad at editing code in a consistent manner, which often leads to more failpoints in the final product if not properly supervised. Additionally, having AI alone write your code, especially during the learning process, means that you are less likely to learn how individual blocks of code work in your overall pipeline. This usually results in an error code that takes much more time to fix because you are less sure how the overall code works. Because you don't get practice, you never get better at the problem. However, AI is incredible at teaching you when prompted correctly. When you run into errors, put the error into AI and ask what the error message means, what line of code

specifically is causing the error, and how you would fix the error. When the AI responds, ask clarifying questions. Remember, LLMs can't judge you, so ask as many times as is necessary to understand how and why the code is working the way it is.

Accessing and Navigating the Terminal

There are a couple of different ways to access the terminal. The first is to click the Linux Mint button in the bottom left and search for the terminal app. For a quicker way of opening the terminal, you can click “Ctrl. + Alt. + T.” Some software available in the “Software Manager” app lets you open and close a terminal using a keybind (I recommend [Guake Terminal](#)). Once you open the terminal, a black box will open that you are able to type commands into.

Note: All blocks of code below will be color coded to the following key:

Input Code

Standard Terminal Output

Error Messages

pwd - Where am I?

While it may not look like it, you are able to navigate the terminal the exact same way that you would navigate the files of your computer. The terminal's equivalent of

folders are called “directories.” When you first open the terminal, you are taken to the “home directory.” This is the default directory that will contain other directories that you make in the future. To make sure that we are where we think we are, **enter the “path to working directory” command into the terminal by typing `pwd` and hitting enter.**

The `pwd` command will output the directory that you are currently looking at. At this point it should look something like `/home/<user>`. Directories are listed like this with each directory being signified with a `/`.

To list all files and directories within your current working directory (the one you’re looking at), use the command:

`ls`

This should output a list of the default directories that come with Linux Mint, such as “`Documents`” and “`Desktop`”.

cd & mkdir - Creating and Moving Through Directories

Data management is an essential skill that will save you countless hours throughout your time as a bioinformatician. For that reason, you typically don’t want to put files directly into your home directory because it will become cluttered very quickly. While everyone will have a different organization system, you should typically create a new directory for each project you start or each application you download.

You can create a directory in a couple of different ways. **The first and most user-friendly way of creating a directory is through the “Files” app.** To create a directory in this manner, open the Files application through the Linux Mint menu (clicking on the bottom left icon when looking at the desktop). This should bring up a file GUI similar to what you would use with Windows or macOS. Right-click on an empty space and click on “Create New Folder.” This is how you would normally create a new directory without using the terminal.

There are many times in which you will need to create and move through directories through either the terminal or in a script you write. **To create a directory in the terminal, use the make directory command by typing `mkdir`.** Try it by typing:

```
mkdir test1
```

To check to see if it worked, list the directories in the working directory by typing:

```
ls
```

If the directory was made, you should see `test1` listed among the output.

Now let's move into the new directory we just created. **To move into a new directory, we can use the “Call Directory Command” by typing `cd` followed by the directory you want to navigate to.** Try it by typing:

```
cd test1
```

Now, you should be in the “test1” directory. Confirm it by using the path to working directory command:

```
pwd
```

If it worked, you should see the following output:

```
/home/<user>/test1
```

Let's make a directory inside this directory and navigate to it by typing:

```
mkdir test2
```

```
cd test2
```

```
pwd
```

You should see the output:

```
/home/<user>/test1/test2
```

In the past few examples, we have been moving “down” directories, or away from the home directory. Now we will be navigating towards the home directory or “up” directories. To go up one directory, we can use two periods. Try it by typing:

```
pwd
```

You should see the output,

```
/home/<user>/test1/test2
```

Now let's navigate up one directory to “test1” by typing:

```
cd ..
```

```
pwd
```

You should see:

```
/home/<user>/test1
```

Now let's do it one more time to get back to the home directory:

```
cd ..
```

```
pwd
```

You should see:

```
/home/<user>
```

Navigating to a directory one step at a time is called “navigating using a relative path.” You can also navigate directly to a chosen directory by calling its “absolute path,” or the exact location it is on your computer. You can think of relative paths as directions; navigating with relative paths is the equivalent of “Take the first left...” while absolute pathing is like inputting the exact address. Navigate to the test2 directory using its absolute path by typing:

```
cd ~/test1/test2
```

```
pwd
```

To which you should see:

```
/home/<user>/test1/test2
```

Notice the “~” in the previous command. This symbol is used in the command line as a shortcut for the home directory. As you may have noticed, the specific user’s name is used in calling the home directory, which would result in issues if a different user were to use a script that references the home directory if we wrote it out verbatim. The ~ shortcut not only saves time but also allows for a script referencing the home directory to reference the home directory of the person running the script.

Oftentimes, you will need to go back to the home directory. Luckily, the home directory can be accessed quickly by typing in `cd` without any path. Try it with:

```
pwd
```

This should output:

```
/home/<user>/test1/test2
```

Then enter:

```
cd
```

```
pwd
```

Which should output your home directory:

```
/home/<user>
```

rm - Removing Directories

If you want to remove a directory through the terminal, use the “remove directory” (`rmdir`) command. However, there are some specific caveats about this command.

Navigate to the home directory, then type:

```
rmdir test1
```

At this point you should see the following error message:

```
rmdir: failed to remove 'test1': Directory not empty
```

Removing a directory or file from the CLI can be dangerous; unlike other OSes, there isn’t a terminal equivalent of a “recycle bin,” meaning that **anything that is deleted can not be recovered**. Because of this, the Bash language that the CLI runs on will always check to see if a directory is empty before running the `rmdir` command. That’s why the error occurred.

There are a couple of ways of getting around this problem. Since the only thing in the “test1” directory is the empty “test2” directory, one way of dealing with this problem is to `cd` into the “test1” directory, then run `rmdir` on “test2” because “test2” is empty, the `rmdir` command would work. Then we `cd` back to the home directory and rerun `rmdir test1`. However, this is a lot of commands and would take a lot of time. Additionally, some directories could be filled with hundreds or thousands of files that would take hours to remove one by one.

An easier and more practical way of removing a non-empty file is to use the “Recursive” and “Force” options for the “Remove” (`rm`) command. **Most commands that you use in the command line will have several different “options” that will slightly modify how a command runs. These options are typically used by adding a hyphen followed by a letter (or letters for multiple options) or a short phrase.** For this case, we can use the “Force Option (`-f`)” combined with the “recursive option (`-r`)” to tell the “`rm`” command to force remove the directory and repeat the command for every item inside that directory. Try it from the home directory with:

```
rm -rf test1
ls
```

If the command worked as expected, you should notice that the “test1” directory no longer exists, and therefore everything contained in the file is also gone. Additionally, a similar option exists for the `rmdir` command with the

`--ignore-fail-on-non-empty` option, but the `rm` command is typically quicker for

this scenario. **Once again, be *extremely careful* with the `rm` and `rmdir` commands.**

If something is deleted like this, you *can not* get it back.

Essential Commands for the Command Line

Now that we know how to orient and move through the terminal, we need to build a vocabulary of essential commands that will be needed for general everyday use of the command line. While these commands aren't specific to bioinformatics, being familiar with these in particular will make your life easier in the long run. Think of it as if you are a doctor practicing in an area with a different common language from yours. You could be the best in the world, but if you don't know how to communicate with your patients (or the files & software) you're working with, it's going to be a struggle.

`sudo` - "No Really, Do It."

I need to admit something: I lied when I said that Linux and the CLI don't have *any* guardrails. By default, Unix will prevent certain actions that are inherently dangerous if accessed by malicious actors. These actions include downloading software that can access your whole system, modifying configuration files that make your computer function, and accessing root files that could brick your computer if edited without care. However, there are times in which you will need to actually do these things (especially download software), and that's where the `sudo` command comes in.

`sudo` stands for “Super User DO” and is the equivalent of an admin password for a Windows device. If you need to do something that the system believes you either don’t have permission to access or that may be detrimental to your computer if you accidentally delete or modify, you are required to put `sudo` in front of the desired command. Once you do this you will be prompted for an administrator password. If you are the administrator of your computer, this will be the password you use for accessing your computer.

Let’s run through an example by trying to list the files in the root directory. Unlike the “home” directory, the root directory contains files that keep your computer up and running:

```
ls /root
```

At this point you will see one of two responses: either a clear error message like,

```
bash: ls /root: Permission denied
```

Or you will be prompted to enter your password and see an output like,

```
[sudo] password for <User>:
```

If your computer defaults to the first option, type `sudo` in front of the command or type in `sudo !!` (`!!` is a shortcut for “run last command again”) and you will see the second prompt. Once you see the second prompt, enter your password to confirm that the computer has permission to make those changes. **When asked for your `sudo` password, always ask yourself if you can trust the program before granting access.**

touch - "Creating a File"

`touch` is the command to create an empty file from the command line. While this can often be done in a more user-friendly way through the files GUI, this command is particularly useful if you want to create a file that you want to output other commands to.

Let's create a test file in a sandbox directory to explore this command as well as some later ones.

```
mkdir sandbox
cd sandbox
touch test.txt
touch test2.txt
ls
```

Which should output:

```
test.txt  test2.txt
```

In the previous series of commands, we made the sandbox directory, traveled into it, made a text document called `test.txt` and listed all the files in the directory.

cat - "Reading and Combining Files"

Technically, the `cat` command is short for "concatenate" and is used to combine two files. While it is really good at that, you'll find yourself using `cat` as an easy way to read files from the terminal. **Let's add some text to the two test files we made earlier and walk through an example:**

```
echo "lorum ipsum" >> test.txt
```



```
echo "is hard to spell" >> test2.txt
```

The previous two commands tell the computer to say back to you – or `echo` – a specific

line of text and store (`>>`) the result of that command (the text you `echoed`) into the

`test.txt` and `test2.txt` files you made earlier. Now let's use `cat` to read the files:

```
cat test.txt
lorum ipsum
cat test2.txt
is hard to spell
```

As you can see, putting `cat` and the file you want to read will result in the output of what

is in the file directly in your terminal. Now, let's use `cat` for its original purpose and

combine the two files.

```
cat test.txt test2.txt >> test3.txt
ls
test.txt  test2.txt  test3.txt
cat test3.txt
lorum ipsum
is hard to spell
```

As you can see the code made a third file called `test3.txt` that contains both of the

text files on different lines. Please note that the `>>` is necessary for the creation of the

new file without that the file will just read both of the files back to back.

mv - Moving and Renaming Files

Now, let's turn the "test3.txt" file into a shell script that can run code. To do this,

the first step is to rename the file to a file ending in ".sh". Similar to the `cat` command

that is used more for reading files than concatenating them, **the move (`mv`) command**

is often used more for renaming rather than moving files. Let's rename the test3 file into a shell file by running the code below:

```
mv test3.txt testScript.sh
ls
test.txt  test2.txt  testScript.sh
```

As you can see, we no longer have the “test3.txt” and instead we now have the `testScript.sh` file. In the case that we wanted to move the directory, we would replace the name of the file name with a path to the directory we want to move it to.

nano - Editing Small Files

If we wanted to edit a file, there are a couple of ways to do it. For larger files, we would want to use a dedicated text editor (See the [Setting Up Essential Software](#) section below). **However, if we want to make a small, simple edit, we can do that directly in the terminal using the `nano` command.** Let's edit the text of the `testScript.sh` file so that it `echoes` the text that's still stored in the file. Let's edit it with the `nano` command:

```
nano testScript.sh
```

This should bring up a terminal text editor with the lines we wrote in the test.txt and test2.txt files. Use the editor to change the file to say `echo "lorum ipsum is hard to spell"`. The bottom of the text editor has a series of different characters preceded by a “^.” the “^” represents either the “Ctrl.” or “Command” key, depending on whether you are on Mac or PC. To exit the text editor, hit `^X` followed by `y` to save changes, then `Enter` to confirm. Finally, `cat` the file to make sure that the changes are saved:

```
cat testScript.sh
```

```
echo "lorum ipsum is hard to spell"
```

If the command returns that output, you are good to go!

chmod - Change Permissions

All of the files that you create will have one or more permissions attached to them. These permissions will either be read (**-r**), write (**-w**), and/or execute (**-x**). These permissions will tell the user (or the programs they run) that they are able to look at, edit, and run in the terminal (if it's code), respectively. If you write a script that you want to run in the terminal and it's not running properly, there's a good chance that you need to change the permissions first. Let's first try to run our new script normally with the following:

```
./testScript.sh
```

The “./” before the script you want to run indicates that you want to execute the script.

However, because we haven't changed the permissions, we should get the following error message:

```
bash: ./testScript.sh: Permission denied
```

To check what permissions the file has, let's use the **ls** command with the **-l** option:

```
ls -l
total 36K
-rw-rw-r-- 1 <owner> <group> 12 Jan 21 12:27 test.txt
-rw-rw-r-- 1 <owner> <group> 17 Jan 21 12:28 test2.txt
-rw-rw-r-- 1 <owner> <group> 36 Jan 21 15:45 testScript.sh
```

While this output looks complicated, it isn't that complicated when you break it down. The first line that's outputted (`total 36K`) shows you the total amount of space that the file takes up in bits. Therefore, `36K` is 36kB of storage taken up by the files in the directory. Each of the next lines represents a file. The first section lists the permissions for the owner, group (a predetermined group of people), and others (everyone else). The second and third parts list the owner of the file and the group they belong to (will be the user if a group isn't assigned). The next two sessions list the file's last edit date and name.

The thing that we want to notice is that the owner can only read and edit (write) the files, but there is no execute privilege for the `testScript.sh` file. To add this file, we will use the `chmod` command with the `+x` option to indicate that we want the file to become executable. Let's add the permission and see how that changes the `ls -l` command.

```
chmod +x testScript.sh
ls -l
-rw-rw-r-- 1 caden caden 12 Jan 21 12:27 test.txt
-rw-rw-r-- 1 caden caden 17 Jan 21 12:28 test2.txt
-rwxrwxr-x 1 caden caden 36 Jan 21 15:45 testScript.sh
```

Notice how the `testScript.sh` file has an `x` in each of the user's permissions. This indicates that the file is now executable for everyone. Test it now by rerunning the code that created the error message:

```
./testScript.sh

lorum ipsum is hard to spell
```

If you got the output seen above, then you know the edits worked!

grep - Finding a Needle in a Haystack

There are times, especially in bioinformatics, when you will want to find something very specific in an extremely large file, directory, or even in the system as a whole. For example, say you have a .fasta file (a file that contains the A's, G's, T's, and C's contained in DNA) and you want to determine the number of times the AAGTCTAG motif appears in that file. In a case like this, you would use the `grep` command.

`grep` stands for Global Regular Expression Print, and this is a command that searches a file, directory, or even your full computer for a specific pattern. To explain this, I have made a completely new file in which I have put the lyrics of Bohemian Rhapsody in.

```
touch rhapsody.txt
```

```
nano rhapsody.txt
```

At this point, I pasted the lyrics into the `nano` editor. Now you can use the `grep`

command to find specific words in the lyrics. For example:

```
grep "no" rhapsody.txt
```

```
Caught in a landslide, no escape from reality
```

```
I'm just a poor boy, I need no sympathy (Ooh, poor boy)
```

```
Put a gun against his head, pulled my trigger, now he's dead
```

There are a lot more outputs, but I will stop there. Notice how the `no`'s are highlighted.

This tool is extremely versatile and has many different options for specific purposes.

See some useful options in **Appendix A** or by using the `man` command.

man - Figuring Out Other Things

There are countless other commands, and with every software you install. If you want to know more about the details, the `man` command is perfect. To use it, simply type `man <command>`, and you will get a detailed description of the command.

Tab Autocomplete and Final Bash Thoughts

I will leave the Bash crash course with one final piece of wisdom. Now that you should be getting comfortable with the command line, tab autocomplete should be extremely useful. Whenever you are typing a command, hit tab to complete. If there is only one option that you can type, the full command will fill out. If not, the command will complete as much as it can. At this point, you can hit the next letter in the command you want to type and hit tab again.

Finally, as a reminder, this is an extremely brief crash course. Additional help would be extremely helpful. For additional resources, I recommend checking out the third-party sources such as those from [CodeAcademy](#), [Command Line Bootcamp](#), or [Unix and Perl Primer for Biologists](#).

Setting Up Essential Software

Now that we are a bit familiar with Bash, we need to download the software needed for proper RNAseq analysis. Three specific pieces of software are essential to

download: R, a text editor meant for code, and Conda.²⁷ Let's first explain what exactly these software are.

Conda

Let's start with Conda. Conda is a software management application that ensures that all of the software that you use is compatible with each other using what is called "Environments." To understand why that's important, we must first consider that software is constantly updated with new syntax, bug fixes, and other changes. Some software also often depends on other preexisting software (often called dependencies). Sometimes, if a software dependency is updated, it could mess up the larger software. While you can probably fix it without too much trouble if it's just a couple of software packages, once you work on larger projects, it would take hundreds of hours to get all of the software packages working for that project. If you are working on multiple projects, this problem grows exponentially.

Conda is a software that solves this problem. Instead of downloading software directly on the computer, you create an environment and install software in that environment using Conda. Conda will make sure that all the software that you install in the environment is compatible with one another and will upgrade/downgrade software and dependencies so that the larger project will work with minimal issues. We can install conda by running the following code:

```
bash anaconda-latest-Linux-x86_64.sh
```

Then we can test if the installation worked with the command:

`conda list`

If a list of packages shows up, then the install was successful.

R and RStudio

Now let's talk about R. R is an open source programming language that is relatively user-friendly and designed specifically for statistics, data analysis, and graphics. R is often used in the sciences as it is relatively user-friendly, has the ability to make publication-ready graphs, and has a lot of backing and support from the wider scientific community.

To download R on your computer, go to <https://cran.rstudio.com/>. From there, click your Linux distro under “Download for Linux” (**Linux Mint will be Ubuntu**). Copy and paste the commands listed into the terminal. To test if R was successfully installed, type “`R`” into the terminal. If you see a message from R appear on the terminal, Congratulations, you have R on your computer! But we’re not done yet; While technically we can do everything we need directly in the R terminal, the experience is even less fun than normally working with the CLI. Instead, we can download what is called an “Integrated Development Environment (IDE)” to make the process much easier. Let’s download an IDE for R called “RStudio.”

To download RStudio, go to the official RStudio download page by Posit at the URL: <https://posit.co/download/rstudio-desktop/>. From there, click the download link for

the Distribution you have (Ubuntu 24 for Linux Mint). Install the program in your home directory. Once that is done, you should be able to find RStudio in the Linux Mint Menu.

Coding Text Editor

You may be familiar with the default text editor (or notepad) that comes with your computer. This is a simple program where you can change the text of a document. Theoretically all your coding can be done in this software. However, text editors that are built to handle code make a world of difference.

There are *many* options for coding specific text editors out there, each with their own specific features and strengths. I recommend doing some research to find the one that works for you, but I prefer [Visual Studio Code](#) (VSCode). This is a popular text editor with support for upgrades and plugins. Before you commit to a text editor, I would highly recommend ensuring that it has a built in linter (a tool that checks your syntax). This is pretty standard but coding without a linter is extremely time consuming and inefficient.

Chapter 2 - Example Script

For the second part of this thesis, we will review a sample pipeline that I wrote to analyze a typical differential expression analysis. First, we will outline a general plan that we hope to accomplish. Then, we will identify how we will accomplish each step of this plan. Some steps we will be able to solve ourselves; however, we will need to use some tools for others. After that, we will explain every step in the thought process of writing the script and build the pipeline step by step. We will then explain how to run and troubleshoot some common errors we may run into.

For this example script, we will be using an mRNA differential expression dataset on the common dog (*Canis lupus familiaris*). Specifically, we will be using the study “RNA-seq of TGF- β induced and histone variant H2A.Z-knocked-down canine MDCK cells in epithelial-to-mesenchymal transition (EMT)” ([ENA Study: ERP022283](#); Study Accession: PRJEB20155)²⁸. This study looked at the role that a specific histone acetylation (H2A.Z) plays on a tumor suppressor gene (TGF- β) and the ability of the affected cancer cells to become malignant (EMT) in an immortalized dog cell line (MDCK). They separated the cells into three different groups: the first group had a gene knockdown for TGF- β , then had shRNA target H2A.Z, the second had shRNA target H2A.Z without having the TGF- β gene knocked out, and the last was a control group that had no treatment assigned to it. mRNA differential expression was used to compare the two treatment groups to the control group, and gene up- and downregulation was determined. Official results can be found on the [Gene Expression Atlas](#).²⁹

I highly recommend that you follow along and build the pipeline yourself as you read this thesis. This will build the skills needed to write future scripts and get you used to writing the syntax needed to code. However, please feel free to use or reference the full script in a GitHub repo that will contain the full script for download.

Outlining a plan

RNAseq Basics

First, let's outline what exactly we are trying to accomplish with RNAseq analysis. Extremely briefly, RNAseq is a technique that is used to figure out what genetic differences cause a specific trait or disease. To do this, we measure the amounts of mRNA we can detect in a subset of cells and use a refseq to identify exactly what genes the mRNA maps to. If we see any genes that are upregulated (producing more mRNA) or downregulated (producing less mRNA), we can associate that gene with the trait or disease we are studying.

Figure 2.1:

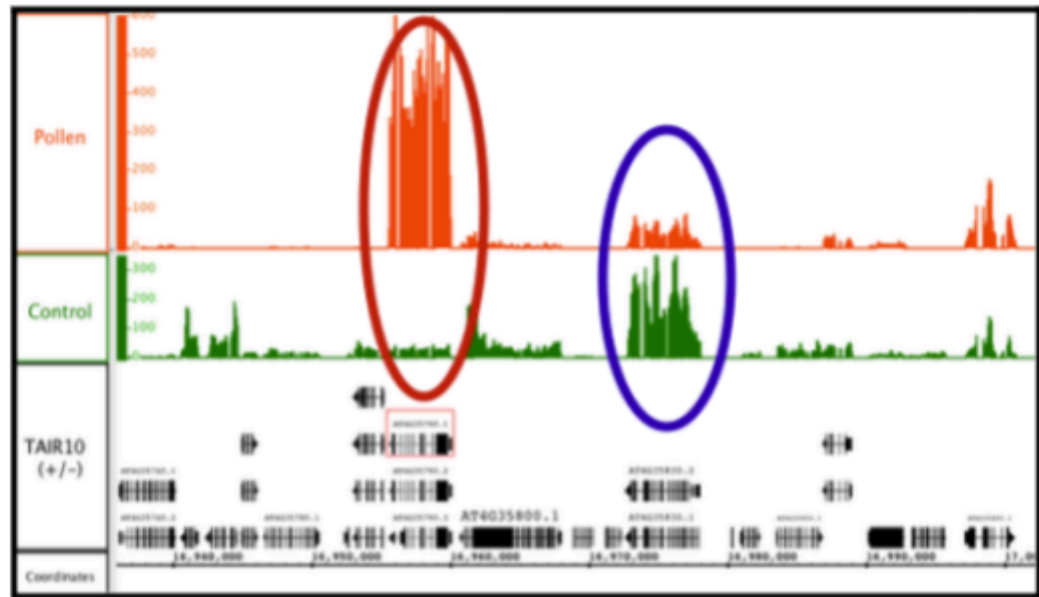


Figure 2.1. Graphical representation of gene up/downregulation of an experimental group (pollen - red) when compared to a control (green). Black bars along the bottom of the plot illustrate the genomic position of transcripts from the *Arabidopsis thaliana* reference genome (TAIR10). The height of the tracks (red and green) illustrate the accumulation of normalized RNA reads within the boundaries of a given gene. The red circle highlights an increase in abundance of RNA molecules in the experimental relative to the control condition. Conversely, the blue circle denotes a decrease in the level of RNA molecules when compared to basal levels.

So now that we know the broad strokes of the direction of where to go, let's figure out what we're going to be working with. First, we need to know what we are working with. Typical RNAseq data will consist of a ".fastq" file that consists of millions of unordered and unorganized short reads that are picked up from the sample. Physically,

this fastq file will be structured like the example following this paragraph. While looking complex, the format is simple. The first line will always start with an @ and contain identifying information for the machine it was run on, sample ID, start time, and other meta information. The second line will consist of the nucleotide sequence. The 3rd line will consist of a single + tying the previous line to the next line. The 4th and final line will consist of a series of characters that represent how confident the sequencer is in saying that the nucleotide above it is correct. Knowing what each character means in its confidence is outside the scope of this thesis, but if you are curious, look up what Phred scores are, then search “ASCII encoded Phred scores.”

Fastq File Example

```
@2fa9ee1... runid=f53ee404... sampleid=18S_am... read=109831 ch=33 start_time=2019-09...
CGGTAGCCAGCTGCGTTCAGTATGGAAGATTTGATTGTTTAGCGATCGCCATACTACCGTGACAAGA
+
+&&-&%$%$%$)$33&0$&%$ ' ' * ' '%$#%$%#+-5/---*&&%$%&() )(&$#&, ' )5769*+..
```

Rough Plan

Now that we know what we'll be working with, let's figure out what we physically need to do. We need to prepare any helper functions we need, set up directories where the files will be going, download the data that we need, build an index file, map our sequences to our index file, count how many maps each gene has, and finally check the quality of our results.

Preparation

The first thing we should do when making a new project is make a dedicated conda environment. Let's create a conda environment called "DE_script" with the "conda-forge" and "bioconda" channels. Type the following command into the terminal:

```
conda create -n DE_script -c conda-forge -c bioconda
```

```
conda activate -n DE_script
```

At this point, you should see the "DE_script" environment activated.

Now that we have the Conda environment, we need to install the tools we'll need to run our pipeline. When you write your own pipeline, you'll find the tools you need either in the planning phase or as you go (if/when things don't go according to plan). However, since we are just following along, we will download all the tools we need now. Once you have activated your conda environment, use the commands in Table 2.1 in the CLI to install the following tools.

Table 2.1: Pipeline Tools & Install

Tool Name	Use case	Install command
NCBI-Datasets-CLI	Download refseqs	<code>conda install ncbi-datasets-cli</code>
HISAT2-Build	Build an index file	<code>conda install hisat2</code>
HISAT2	Map reads to refseq	<code>conda install hisat2</code>
featureCounts	Count reads mapped to specific genes	<code>conda install subread</code>
MultiQC	Identify the quality of the sample and pipeline	<code>conda install multiqc</code>
Pandas	Python package for data manipulation	<code>Conda install pandas</code>
r	A statistical analysis software	<code>Conda install r</code>
rpy2	A package that bridges python and R	<code>conda install rpy2</code>

Table 2.1. A table of pipeline tools that are needed to run the pipeline as well as code to install each of the tools in a conda environment

A couple of important things to note are that both HISAT2 and its corresponding build tool are both packaged under the `hisat2` package, meaning you won't need to run that code twice. Additionally, if you try to install featureCounts directly, your installation will fail. The tool is located in the `subread` software package.

We will be running our script in Python which means that we will need to import some useful packages of library functions that the open source community has made.

We can use the python package installer `pip` to do this but we need to install it first.

Let's do this by using the command:

```
conda install pip
```

Then accept any confirmation that shows up.

Now that we have `pip` installed on our environment, we need to import the packages that have helper functions we will be using in our script. When you are building a script on your own, you will install these packages as you need them. However, for simplicity we will download all the packages we will need right now. Make sure that you have activated the Conda environment and then install the software according to [Table 2.1](#).

Finally we need to make a table listing the experiment's "metadata." This includes important information about our data and we are most interested in making sure that we label our experimental and control groups. Additionally, it is typically best practice to put these files in a designated folder. To do this let's start on the script. First, let's create a new working directory called "diffEx" and `cd` into it. Use the `mkdir inputFiles` to create a directory to hold our metadata. Finally, use a table editor like google sheets to make a `metadata.csv` file formatted like the table in Table 2.2.

Table 2.2: Metadata.csv Format

SampleID	Condition
ERR1899753	Control
ERR1899755	Control
ERR1899757	Control
ERR1899758	Experimental_1
ERR1899760	Experimental_1
ERR1899762	Experimental_1
ERR1899763	Experimental_2
ERR1899765	Experimental_2
ERR1899767	Experimental_2

Table 2.2. A simple table showing how the metadata.csv table is supposed to be formatted.

Now that we've prepared our Conda environment and metadata. We can finally start on our script. In the project directory, make a new empty file called "deScript.py" and open it in your text editor.

Import Libraries - Gather Your Tools

```
2  import argparse
3  from ast import If
4  import json
5  import subprocess
6  import os
7  import sys
8  import glob
9  import pandas as pd
10 import zipfile
11 import shutil
12 import datetime
13 import time
14
15
16 if not os.environ.get("_RPY2_ENV_FIXED"):
17     r_home = subprocess.run(["R", "RHOME"], capture_output=True,
18                             text=True).stdout.strip()
19     if r_home:
20         r_lib = os.path.join(r_home, "lib")
21         ld_dirs = [d for d in os.environ.get("LD_LIBRARY_PATH", "").
22                     split(":") if d]
23         os.environ["R_HOME"] = r_home
24         os.environ["LD_LIBRARY_PATH"] = ":".join([r_lib] + [d for d
25                                                         in ld_dirs if d != r_lib])
26         os.environ["_RPY2_ENV_FIXED"] = "1"
27         os.execve(sys.executable, [sys.executable] + sys.argv, os.
28                 environ)
29
30 import rpy2
31 import rpy2.robj as robj
32 from rpy2.robj.packages import importr, data
33 import rpy2.robj.packages as rpackages
```

The first step in any Python script is to import any functions from Python libraries we will need to use in our script. Python comes installed with a lot of helpful functions that make our lives a lot easier in specific situations. However, many of these functions

are situational, and the python community has developed additional ones to help with pretty much anything you could imagine.

We need to tell Python to look at some of these specific libraries using the `import` command. When you are making your pipeline this section will start small and, you will add more `imports` as you need them.

You may notice that the `rpy2` library has an `if` statement preceding it. The `rpy2` library acts as a bridge between R and Python. Because of this, we need to make sure that Python knows where R is before the `main` part of the script runs. This `if` statement checks to see if the R library is visible by Python. If it isn't, it prepares some variables then restarts the script in preparation for the rest of the script.

Creating Helper Functions

The next thing we do when making a bioinformatics pipeline (or anything) in Python is make helper functions that will assist in writing the main part of the script. These helper functions are usually made to either assist you with debugging by letting you know where and how the script failed or by turning a long and repetitive command into a shorter one. We will be making three helper functions with our script. The first will be a command that will help with errors. More specifically, we want to tell Python to run a tool in BASH and physically tell us when the program runs and if it completes. That

way, if it fails, we are able to pinpoint exactly where it failed without having to translate the error logs. Put the full command below, then we'll go over each part of it.

Run Command Helper Function

```
33 def run_command(command, description):
34     """
35     Helper function to run shell commands with error handling.
36     """
37     print(f"\n[STATUS] Starting: {description}")
38     print(f"Command: {' '.join(command)}")
39     try:
40         subprocess.run(command, check=True)
41         print(f"[SUCCESS] Completed: {description}")
42     except subprocess.CalledProcessError as e:
43         print(f"[ERROR] Failed during: {description}")
44         print(f"Error details: {e}")
45         sys.exit(1)
46     except FileNotFoundError:
47         print(f"[ERROR] Executable not found for command: {command[0]}")
48         print("Please ensure the tool is installed and in your PATH.")
49         sys.exit(1)
```

```
33 def run_command(command, description):
```

The `def` in the first line tells Python that the following text is a function. The `run_command` is the name that you give your function, and the `command` and `description` within the parentheses are parameters that you will reference in the function. In order to run this command later on in the script, we will make a list of the commands we would type into the terminal, then pass that list into the `command`

parameter. For example, if we wanted to run the command `cd && ls -a` we would store that command as a list in Python like this:

```
cd_cmd = ["cd", "&&", "ls -a"]
```

Then, when we want to run the command, we would type this into the script:

```
run_command(cd_cmd, "Go to home directory then list all files & directories")
```

```
34     """
35     Helper function to run shell commands with error handling.
36     """
```

This part of the script is a comment and actually doesn't do anything. In Python, anything that has either a `#` in front of it, or is enclosed by `"""` is treated as a comment and is ignored by the program. The main purpose of this is to remind you (the programmer) what the section does so that you don't need to decipher the code after not looking at it for a while.

```
37     print(f"\n[STATUS] Starting: {description}")
38     print(f"Command: {' '.join(command)}")
```

This part of the code will `print` or display directly on the screen a short string of your choosing the `f` in front of the `"` tells Python to treat anything inside `{ }` as if it weren't part of the string. This means that the terminal will tell you that it's starting whatever task you enter as the description. The second print line will tell you the exact command that is being run. Since we will be storing our `command` as a list. The `'`

`'.join(command)` will be how we connect that list into a command that the terminal can use. This command says that the `command` list we have will be `.joined` with a space (`' '`). For our example, `cd_cmd` the join process would be joined together like this `cd && ls -a`.

```
39     try:
40         subprocess.run(command, check=True)
41         print(f"[SUCCESS] Completed: {description}")
```

This part of the code will `try` to tell the `run` command from the `subprocess` library to run the list you stored in the `command` list. The `subprocess.run` command expects a list in the same format as was pieced together with the previous `.join` command. The `check=True` parameter tells Python to check to see if it completed successfully. If not, it'll call a separate `subprocess` function called `CalledProcessError` that will let you know what went wrong. If the command ends successfully, then the rest of the function is skipped. If the process fails, then we will go to the next part of the function.

```
42     except subprocess.CalledProcessError as e:
43         print(f"[ERROR] Failed during: {description}")
44         print(f"Error details: {e}")
45         sys.exit(1)
```

The `except` statement will only occur if the first process fails. If this occurs, Python will call the `CalledProcessError` function from the `subprocess` library that is automatically created with the `check=True` parameter in the previous section and store it as the variable `e`. It will `print` that the command failed, similarly to the success process of the previous section, then tell you the error details by printing whatever was inside `e`. The function will then exit using `sys.exit(1)` which tells Python that an error has occurred.

```
46     except FileNotFoundError:
47         print(f"[ERROR] Executable not found for command: {command[0]}")
48         print("Please ensure the tool is installed and in your PATH.")
49         sys.exit(1)
```

This last part of the function will only occur if the previous two sections didn't work and is functionally similar to the previous section. However, as you may be noticing, the previous two scenarios were "If the `subprocess` command worked" or "if it didn't work." The only other situation that may occur is if the `subprocess` command wasn't found at all. Since the package comes with Python, this is unlikely, but it's good to cover it just in case.

Check Tools Helper Function

```
54 def check_tools(tools):
55     """
56     Verify that necessary tools are installed and available in PATH.
57     """
58     missing = []
59     for tool in tools:
60         if shutil.which(tool) is None:
61             missing.append(tool)
62
63     if missing:
64         print(f"[ERROR] The following tools are missing from your
65             PATH: {' '.join(missing)}")
66         print("Please install them (e.g., via Conda) before running
67             this script.")
68         sys.exit(1)
```

Next, we will look at the `check_tools` helper function. This helper function is designed to check your conda environment to see if you have all the tools you need installed. If it detects that some are missing, it will end the program immediately. It's helpful to know it immediately because if not, you will spend hours waiting for a program to finish just to have it fail. **Additionally, this and future code will have similar parts to the previous code. The full code is available, but I will only explain the new code in detail (I will refer back to the previous code whenever possible).**

Later in the main portion of the pipeline, you will make a list of tools that you will need to pre-install on conda. By passing those tools into the `tools` parameter, you can ensure *that the program* can see the tools before moving on. Even if you know you have the tools installed, it's always good to have something like this so that you know that nothing is going wrong.


```
58     missing = []
```

This first line makes an empty list that the function can refer to later on.

```
59     for tool in tools:
60         if shutil.which(tool) is None:
61             missing.append(tool)
```

This next section is a `for` loop that will look at each item (which we call `tool`) that we put in the `tools` parameter. **Note that whether or not it's plural is important; each `tool` is a single element of `tools`.** Since we will be putting a list in the `tools` parameter, the `for` loop will run the indented text below it on every item in the list. The `if` statement will use a package library called `shutil` to run subprocesses on your computer terminal. In this case, it'll run the `which` command to see if the `tool` it is looking at is present on your computer. If the tool isn't installed on your conda environment, the `which` command will not return anything (`None`) and add that tool to the `missing` list.

```
63     if missing:
64         print(f"[ERROR] The following tools are missing from your
        PATH: {' '.join(missing)}")
65         print("Please install them (e.g., via Conda) before running
        this script.")
66         sys.exit(1)
```

If any tools were added to the `missing` list, the function will list those tools to you, similarly to the first `except` of the `run_command` function.

Find Fastq Pairs Helper Function

```
70 def find_fastq_pairs(directory):
71     """
72     Scans a directory for paired fastq files matching the pattern
73     *_1.fastq* and *_2.fastq*.
74     Returns a list of tuples: [(sample_name, r1_path, r2_path), ...]
75     """
76     pairs = []
77     # Search for forward reads (supports .fastq, .fq, .fastq.gz, .fq.
78     # gz)
79     # Using a broad pattern to catch the _1 marker
80     forward_files = glob.glob(os.path.join(directory, "*_1.fastq*"))
81     + \
82     glob.glob(os.path.join(directory, "*_1.fq*"))
83
84     for r1 in forward_files:
85         # Determine the expected reverse file name
86         # This replaces the last occurrence of _1 with _2
87         base_name = os.path.basename(r1)
88
89         # Simple string replacement for the pairing suffix
90         if "_1.fastq" in r1:
91             r2 = r1.replace("_1.fastq", "_2.fastq")
92             sample_id = base_name.split("_1.fastq")[0]
93         elif "_1.fq" in r1:
94             r2 = r1.replace("_1.fq", "_2.fq")
95             sample_id = base_name.split("_1.fq")[0]
96         else:
97             continue # specific extension not matched logic above,
98             skip
99
100         if os.path.exists(r2):
101             pairs.append((sample_id, r1, r2))
102         else:
103             print(f"[WARNING] Found forward read {base_name} but no
104             matching reverse read found at {r2}. Skipping.")
105
106     return sorted(pairs)
```

When you download fastq files from the internet or from a sequencer, you will often get both forward and reverse reads. It is standard naming convention to have the

run name at the front of the file, followed by either a 1 or 2 to indicate if it's that run's forward or reverse read. When you provide a directory, this helper function is meant to automatically organize these files so that you can accurately pair them together when needed.

```
75     pairs = []
76     # Search for forward reads (supports .fastq, .fq, .fastq.gz, .fq.
77     # Using a broad pattern to catch the _1 marker
78     forward_files = glob.glob(os.path.join(directory, "*_1.fastq*"))
79     + \
       glob.glob(os.path.join(directory, "*_1.fq*"))
```

The first line makes an empty list called `pairs`. Then it makes a list of file paths and saves it as the variable `forward_files`. In order to do this, it uses the `glob` function from the `glob` package that allows us to use wildcards to add multiple files without having to know the exact file names. It also uses the `os.path.join` function from the `os` module to make sure that the file path can work regardless of the OS they are using. The code goes into the directory you put into the `parameter` and checks for each file that has either a `1.fastq` or `_1.fq` in it and saves its directory to the `forward_files` list.

```
81     for r1 in forward_files:
82         # Determine the expected reverse file name
83         # This replaces the last occurrence of _1 with _2
84         base_name = os.path.basename(r1)
```

This `for` loop goes through each of the forward file directories in the `forward_files` list and uses the `os.path.basename` to take the file name and saves it to a variable named `base_name`.

```
86         # Simple string replacement for the pairing suffix
87         if "_1.fastq" in r1:
88             r2 = r1.replace("_1.fastq", "_2.fastq")
89             sample_id = base_name.split("_1.fastq")[0]
90         elif "_1.fq" in r1:
91             r2 = r1.replace("_1.fq", "_2.fq")
92             sample_id = base_name.split("_1.fq")[0]
93         else:
94             continue # specific extension not matched logic above,
                        skip
```

This `if` statement will take the forward read (saved as `r1`) that the computer is looking at, replace the `_1.fastq` in the basename with a `_2.fastq`, and save it to the variable `r2`. It will then save the text before the `"_1.fq"` (the run ID) to a variable called `sample_id`. If the file ends with a `_1.fq` instead, the `elif` statement will do the same with that naming convention. If the file doesn't fit either of these, the `else` statement will skip this step entirely.

```
96         if os.path.exists(r2):
97             pairs.append((sample_id, r1, r2))
98         else:
99             print(f"[WARNING] Found forward read {base_name} but no
100                  matching reverse read found at {r2}. Skipping.")
101     return sorted(pairs)
```

This `if` statement will check to see if the `r2` path exists (the file with “_2”). If the file exists, it will `append` the run ID (`sample_id`), forward file path (`r1`), and reverse file path (`r2`) to the `pairs` list we made previously. We will refer to this later on in the script. If it doesn't, then it will trigger the `else` statement, letting you know that it didn't find a file but will try to continue.

Main Function - Why is it There?

At this point, we have created all the helper functions that we need, meaning that we can move to the actual program that your computer will be running. You may notice that this part of the script is stored under a function called `main`. Because it's a function, this means it won't be called immediately when we run the script. However, if we skip to the bottom of the script, we see what actually runs when we call the script is this:

```
747 if __name__ == "__main__":  
748     main()
```

When you run a Python script in the terminal, Python assigns the string “`__main__`” to a secret variable called `__name__`. However, if you import the script as we do with `subprocess`, `glob`, etc, it'll save the base name (the filename without the “.py”) to `__main__`. This means, in our code, essentially says, if `__main__` is `__main__`, run the program. While this seems unnecessary, it allows you to import this script into another Python script without it messing up the script you are importing it into. This is actually common practice when you write a Python script, and is highly recommended.

Let's use an example to explain why this is important. Say we hypothetically make another script, but we want to reuse the `find_fastq_pairs` helper function. Instead of copying and pasting the function into our new script, we can instead `import deScript` and then run `deScript.find_fastq_pairs(directory)` in our new script. Without the `def main()`, the function would see the “`deScript...`” at the front of the command and run the full script within our script, even though we only want to use the helper function. **With this in mind, it is highly suggested to use this syntax in any Python script you use.**

Creating Arguments

```
112     script_start_time = datetime.datetime.now()
113
114
115     print(f"Script Start Time: {datetime.datetime.now().strftime
116           ("%Y-%m-%d %H:%M:%S")}")
117
118     parser = argparse.ArgumentParser(description="Automated RNA-Seq
119     Pipeline: Download -> Index -> Align -> Count -> QC -> R
120     Analysis")
121
122     # Required arguments
123     parser.add_argument("-a", required=True, help="NCBI RefSeq
124     Assembly Accession (e.g., GCF_011100685.1 for Canis lupus
125     familiaris)")
126     parser.add_argument("-fq", required=True, help="Directory
127     containing paired FASTQ files (e.g., sample_1.fastq.gz)")
128
129     # Optional arguments
130     parser.add_argument("--threads", default="10", help="Number of
131     threads to use (default: 10)")
132     parser.add_argument("--output_dir", default="rnaseq_output",
133     help="Directory for output files")
134     parser.add_argument("--keep_zip", action="store_true",
135     help="Keep the downloaded NCBI zip file")
136
137     args = parser.parse_args()
```

This section of the script is meant to allow you to add arguments to the script. Arguments are functionally identical to parameters for functions or options for BASH commands. We will explain each section in more detail below.

```
112     script_start_time = datetime.datetime.now()
```


This section of the script simply saves the time the command was run to a `script_start_time` variable and `prints` that time to the screen. This serves no functional purpose for the script but is used to keep track of where the script is and how long it has been running. **There are many more instances of code like this in the future, and the explanation for which will be skipped.**

```
117     parser = argparse.ArgumentParser(description="Automated RNA-Seq  
Pipeline: Download -> Index -> Align -> Count -> QC -> R  
Analysis")
```

This code makes a variable named `parser` that holds the information needed for our arguments. The code uses the `ArgumentParser` function from the `argparse` library with the `description` parameter with a brief explanation of the pipeline. This description will show when someone uses the `-h` or `--help` argument, along with all the possible arguments and their `help` parameters.

```
119     # Required arguments  
120     parser.add_argument("-a", required=True, help="NCBI RefSeq  
Assembly Accession (e.g., GCF_011100685.1 for Canis lupus  
familiaris)")  
121     parser.add_argument("-fq", required=True, help="Directory  
containing paired FASTQ files (e.g., sample_1.fastq.gz)")
```

This section of code sets the required arguments that are needed in order for the code to run. If these arguments aren't included when running the script, the script will exit before running. The first parameter is the name of the argument, the second

parameter states if it is required, and the third is a brief description that will appear if the user uses the `-h` argument on the script. The first argument asks the user for the NCBI accession number for the RefSeq we want to use. The second asks for the directory for the fastq files you want to analyze.

clone

```
125     # Optional arguments
126     parser.add_argument("--threads", default="10", help="Number of
127         threads to use (default: 10)")
128     parser.add_argument("--output_dir", default="rnaseq_output",
129         help="Directory for output files")
130     parser.add_argument("--keep_zip", action="store_true",
131         help="Keep the downloaded NCBI zip file")
132
133     args = parser.parse_args()
```

The next set of arguments is the optional arguments. They work identically to the required arguments with the exception that the `required=True` is replaced with `default=<something>`. If the argument is left blank, any instance of that argument will be replaced with what was set as the default. The first of these optional arguments sets the number of PC threads that will be dedicated to the pipeline. Threads are parts of your PC that are dedicated to completing tasks. The second argument is `--output_dir`, which creates a directory to put all of the created files in. The third option, `--keep_zip`, gives you the option to keep the zip file that the refseq comes from (on by default).

Pre-Flight Check

```
135 # 0. Pre-flight check
136 preflight_start_time = datetime.datetime.now()
137 print(f"Preflight Start: {datetime.datetime.now().strftime(
138     \"%Y-%m-%d %H:%M:%S\")}")
139 print(f"Run Time: {preflight_start_time - script_start_time}
140     \n\n")
141
142 required_tools = ["datasets", "hisat2-build", "hisat2",
143     "featureCounts", "multiqc", "fastqc"]
144 #, "rpy2", "r"
145 check_tools(required_tools)
146
147 preflight_end_time = datetime.datetime.now()
148 print(f"Preflight End: {datetime.datetime.now().strftime(
149     \"%Y-%m-%d %H:%M:%S\")}")
150 print(f"Run Time: {preflight_end_time - script_start_time}\n\n")
```

This code is what we call a pre-flight check; in this code, we make sure that the computer has the required tools needed for the pipeline. If the tools aren't there, you don't want to waste time and computing power by running a pipeline that will inevitably fail.

```
140 required_tools = ["datasets", "hisat2-build", "hisat2",
141     "featureCounts", "multiqc", "fastqc"]
142 #, "rpy2", "r"
143 check_tools(required_tools)
```

In this code, we make a list called `required_tools` that consist's of all the tools we will need for the pipeline. These are the tools that we installed in the

["Preparation"](#) section and uses the `check_tools` function that we described in the ["Creating Helper Functions"](#) section.

Verify Fastq Files

```
149 # 1. Verify FASTQ inputs
150 ver_fastq_start_time = datetime.datetime.now()
151 print(f"fq Verification Start: {datetime.datetime.now().strftime(
152     ("%Y-%m-%d %H:%M:%S"))}")
153 print(f"Run Time: {ver_fastq_start_time - script_start_time}
154     \n\n")
155
156 if not os.path.isdir(args.fq):
157     print(f"[ERROR] FASTQ directory not found: {args.fq}")
158     sys.exit(1)
159
160 sample_pairs = find_fastq_pairs(args.fq)
161 if not sample_pairs:
162     print(f"[ERROR] No paired FASTQ files found in {args.fq}
163         matching pattern *_1.fastq*/*_2.fastq*")
164     sys.exit(1)
165
166 print(f"[STATUS] Found {len(sample_pairs)} sample pairs to
167     process:")
168 for s, r1, r2 in sample_pairs:
169     print(f"    - {s}")
170
171 ver_fastq_end_time = datetime.datetime.now()
172 print(f"fq Verification End: {datetime.datetime.now().strftime(
173     ("%Y-%m-%d %H:%M:%S"))}")
174 print(f"Run Time: {ver_fastq_end_time - script_start_time}\n\n")
```

In this section of the code, we will verify that the directory containing the fastq files are formatted in a way that the pipeline is able to be used. If the program isn't able to parse the data using the `find_fastq_pairs` helper function we [previously described](#), the program will exit.

```

154     if not os.path.isdir(args.fq):
155         print(f"[ERROR] FASTQ directory not found: {args.fq}")
156         sys.exit(1)

```

This if statement will use the `isdir` (“is directory?”) function in the `os` module (path submodule) to see if the path we used for the argument `--fastq_dir` is even a directory. If the user used a file or misspelled the directory, the script will `print` that it couldn’t find the directory and exit. If it passes, the next block of code will run.

```

158     sample_pairs = find_fastq_pairs(args.fq)
159     if not sample_pairs:
160         print(f"[ERROR] No paired FASTQ files found in {args.fq}
161             matching pattern *_1.fastq*/_2.fastq*")
162         sys.exit(1)

```

This next section will first run the `find_fastq_pairs` helper function on what the user entered as the `--fastq_dir` and save the outputted list (in [ID, path to forward, path to reverse] format) and save it to the variable `sample_pairs`. If the directory doesn’t contain any fastq files or those fastqs aren’t formatted properly, the following `if` statement will cause the terminal to `print` an error message and exit safely (“`sys.exit(1)`”).

```

163     print(f"[STATUS] Found {len(sample_pairs)} sample pairs to
164         process:")
165     for s, r1, r2 in sample_pairs:
166         print(f"    - {s}")

```

If those last two safety checks passed, then this code will tell the user that it found the specific number of files to process, and then print out the ID of the samples found. One important thing I would like to bring attention to is the structure of the `for` loop. Most `for` statements will be followed by one variable leading to every item being treated the same. For our example, it's followed by three variables. Because of this, the first item in the list will be treated as the variable `s`, the second `r1`, and the third `r2` it will then repeat and treat the fourth as `s` again. This makes it so that if we have a list that follows a specific pattern we can choose only to work with a subset of that list. Because our example works with a list that is formatted as [ID, forward read, reverse read] we can choose to only print the ID or `s`.

Setup Output Directories

```
172 # Setup directories
173 setup_dir_start_time = datetime.datetime.now()
174 print(f"Setup Dir Start: {datetime.datetime.now().strftime
175      (\"%Y-%m-%d %H:%M:%S\")}")
176 print(f"Run Time: {setup_dir_start_time - script_start_time}
177      \n\n")
178
179 os.makedirs(args.output_dir, exist_ok=True)
180 genome_dir = os.path.join(args.output_dir, "genome_data")
181 counts_dir = os.path.join(args.output_dir, "counts")
182 align_dir = os.path.join(args.output_dir, "alignments")
183 qc_dir = os.path.join(args.output_dir, "qc")
184 multiqc_dir = os.path.join(args.output_dir, "qc",
185      "multiqc_report")
186 fastqc_dir = os.path.join(args.output_dir, "qc", "fastqc_report")
187 os.makedirs(qc_dir, exist_ok=True)
188 os.makedirs(fastqc_dir, exist_ok=True)
189 os.makedirs(multiqc_dir, exist_ok=True)
190 os.makedirs(counts_dir, exist_ok=True)
191 os.makedirs(align_dir, exist_ok=True)
```

At this point, we have confirmed that the user has everything they need to run the pipeline. The next step is to prepare the output directory to hold the files made by the pipeline in an organized way.

```
177 os.makedirs(args.output_dir, exist_ok=True)
```

This specific line of code checks to see if the user specified a specific `output_dir` as an argument. If they don't, the default of `rnaseq_output` that was specified in the [“Creating Arguments”](#) section will be entered. The `exists_ok=True`

parameter lets the program know that if the directory already exists (which would normally cause an error), the program will continue using that directory.

```
179 | counts_dir = os.path.join(args.output_dir, "counts")
```

```
187 | os.makedirs(counts_dir, exist_ok=True)
```

These next two lines of code are technically separated, but are together here to save space. The first line of code will make a new subdirectory inside our `output_dir` called `genome_data`. If a subdirectory already exists with that name, the next line of code will use that directory instead. This code is repeated for a `counts_directory`, `align_dir`, `sam_dir`, and `qc_dir`.

Download Genome Data

```
198 # 2. Download Genome Data
199 genome_download_start_time = datetime.datetime.now()
200 print(f"Genome Download Start: {datetime.datetime.now().strftime(
201     ("%Y-%m-%d %H:%M:%S"))}")
201 print(f"Run Time: {genome_download_start_time -
202     script_start_time}\n\n")
202 zip_filename = os.path.join(args.output_dir, "ncbi_dataset.zip")
203
204 # Only download if we don't already have the extracted data
205 if not os.path.exists(genome_dir):
206     download_cmd = [
207         "datasets", "download", "genome", "accession", args.a,
208         "--include", "gtf,genome", # genome implies fasta
209         "--filename", zip_filename
210     ]
211     run_command(download_cmd, "Downloading RefSeq Genome and
212         GTF")
212
213     print(f"[STATUS] Unzipping {zip_filename}...")
214     try:
215         with zipfile.ZipFile(zip_filename, 'r') as zip_ref:
216             zip_ref.extractall(genome_dir)
217     except zipfile.BadZipFile:
218         print("[ERROR] Downloaded file is not a valid zip file.")
219         sys.exit(1)
220
221     if not args.keep_zip:
222         os.remove(zip_filename)
223 else:
224     print(f"[STATUS] Genome directory {genome_dir} exists.
225         Skipping download.")
226
227 genome_download_end_time = datetime.datetime.now()
228 print(f"Genome Download End: {datetime.datetime.now().strftime(
229     ("%Y-%m-%d %H:%M:%S"))}")
229 print(f"Run Time: {genome_download_end_time - script_start_time}
230     \n\n")
```

Once again, in RNASeq we need to prepare a refseq to compare our results to.

Most of these refseqs can be found online through a genome browser. Some useful

browsers that will have most general and well known genomes that you will need for most projects include the [NCBI](#)³⁰, [UCSC](#)³¹, and [Ensembl](#)³². For some more common model organisms there will be specialized genome ontologies with more specialized datasets (For example [FlyBase](#)³³ is a gene ontology for *Drosophila melanogaster*).

```
202 | zip_filename = os.path.join(args.output_dir, "ncbi_dataset.zip")
```

This specific line of code will see if the `ncbi_dataset.zip` already exists in the working directory. If the file exists, it will save it to a variable called `zip_filename`.

```
221 |         if not args.keep_zip:
222 |             os.remove(zip_filename)
223 |     else:
224 |         print(f"[STATUS] Genome directory {genome_dir} exists.
        Skipping download.")
```

This section will check if the `keep_zip` argument was set to true (It is by default). If the `keep_zip` argument was set to `False`, then the `remove` function from the `os` package will remove the zip file located in the `zip_filename` variable. If the zip file already exists when creating the `zip_filename` variable, the screen will let you know that the file already exists and that it will skip the next command.

```

204     # Only download if we don't already have the extracted data
205     if not os.path.exists(genome_dir):
206         download_cmd = [
207             "datasets", "download", "genome", "accession", args.a,
208             "--include", "gtf,genome", # genome implies fasta
209             "--filename", zip_filename
210         ]
211         run_command(download_cmd, "Downloading RefSeq Genome and
GTF")

```

If the genome directory doesn't exist, then the following line of code runs. Since the command actually runs in BASH using the `run_command` function we first need to make a list that will be combined. We want our final command to be,

```
datasets download genome accession <accession number> --include
gtf,genome --filename <zip file name>
```

So we enter each of those words each separated by a comma and enclosed in " ". There are two parameters that are not enclosed with " ", which are arguments that the user will input when they run the script. The line starting with `run_command...` will string the command together to run in the command in whatever language the computer is using (BASH for our case) then print "Downloading Refseq Genome and GTF" *.

The `try` loop will first print that the zip file will be unzipped. Then, the program will unzip the file located at the `genome_dir` directory using the `zipFile` package. The `except` statement will detect if there are any problems with the zipfile and print an

error message on the screen before exiting. If the `keep_zip` argument was set to `False`, the `remove` function from the `os` package will remove the zip file. If the `genome_dir` directory exists, the `else` statement will trigger, causing the script to skip the previously described steps.

Locate FASTA and GTF

```
233 # Locate FASTA and GTF files
234 locate_genome_files_start_time = datetime.datetime.now()
235 print(f"Locate Genome Files Start: {datetime.datetime.now().
    strftime("%Y-%m-%d %H:%M:%S")}")
236 print(f"Run Time: {locate_genome_files_start_time -
    script_start_time}\n\n")
237
238 search_path = os.path.join(genome_dir, "ncbi_dataset", "data",
    args.a)
239 fasta_files = glob.glob(os.path.join(search_path, "*.fna")) +
    glob.glob(os.path.join(search_path, "*.fasta"))
240 gtf_files = glob.glob(os.path.join(search_path, "*.gtf"))
241
242 if not fasta_files:
243     print(f"[ERROR] No FASTA (.fna/.fasta) file found in
        {search_path}")
244     sys.exit(1)
245 if not gtf_files:
246     print(f"[ERROR] No GTF file found in {search_path}")
247     sys.exit(1)
248
249 ref_fasta = fasta_files[0]
250 ref_gtf = gtf_files[0]
251
252 print(f"Found FASTA: {ref_fasta}")
253 print(f"Found GTF: {ref_gtf}")
254
255 locate_genome_files_end_time = datetime.datetime.now()
256 print(f"Locate Genome Files End: {datetime.datetime.now().
    strftime("%Y-%m-%d %H:%M:%S")}")
257 print(f"Run Time: {locate_genome_files_end_time -
    script_start_time}\n\n")
```

This next section is focused on making some variables that will store the directory of the FASTA and GTF files within our downloaded zipfile.

```
238     search_path = os.path.join(genome_dir, "ncbi_dataset", "data",
239                                args.a)
239     fasta_files = glob.glob(os.path.join(search_path, "*.fna")) +
240                     glob.glob(os.path.join(search_path, "*.fasta"))
240     gtf_files = glob.glob(os.path.join(search_path, "*.gtf"))
```

The extracted zipfile will contain a folder that is structured in a standardized format. The more upstream directories will contain metadata and reports for the downloaded data and our requested will always be located in the `ncbi_datasets/data` subdirectory.

The first few lines make a `search_path` directory that holds the important `ncbi_datasets/data` subdirectory, as well as `fasta_files` and `gtf_files` lists that will hold paths to any file in the `search_path` directory that ends in either `".fna"` / `".fasta"` or `".gtf"` respectively.

```
242     if not fasta_files:
243         print(f"[ERROR] No FASTA (.fna/.fasta) file found in
244               {search_path}")
244         sys.exit(1)
245     if not gtf_files:
246         print(f"[ERROR] No GTF file found in {search_path}")
247         sys.exit(1)
```

If there are no FASTA or GTF files in the expected location, one of these lines of code will recognize it, let the user know with an error message, and then stop safely.

```
249     ref_fasta = fasta_files[0]
250     ref_gtf = gtf_files[0]
251
252     print(f"Found FASTA: {ref_fasta}")
253     print(f"Found GTF: {ref_gtf}")
```

In nearly every case there will only be one FASTA and GTF file to reference. However, in the case that there are multiple, the reference FASTA and GTF will be the first file in the list. This line of code will assign that first file to a variable called `ref_fasta` or `ref_gtf` and let the user know. These variables will be used in future steps of the script.

Build HISAT2 Index

```
259 # 3. Build HISAT2 Index
260 hisat2_index_build_start_time = datetime.datetime.now()
261 print(f"HISAT2 Index Build Start: {datetime.datetime.now().\nstrftime('%Y-%m-%d %H:%M:%S')}")
262 print(f"Run Time: {hisat2_index_build_start_time -\nscript_start_time}\n\n")
263
264 index_prefix = os.path.join(genome_dir, f"{args.a}_index")
265 if not os.path.exists(f"{index_prefix}.1.ht2"):
266     build_cmd = [
267         "hisat2-build",
268         "-p", args.threads,
269         ref_fasta,
270         index_prefix
271     ]
272     run_command(build_cmd, "Building HISAT2 Index")
273 else:
274     print("[STATUS] HISAT2 index found. Skipping build.")
275 hisat2_index_build_end_time = datetime.datetime.now()
276 print(f"HISAT2 Index Build End: {datetime.datetime.now().strftime\n('%Y-%m-%d %H:%M:%S')}")
277 print(f"Run Time: {hisat2_index_build_end_time -\nscript_start_time}\n\n")
```

The following block of code will build what is called a HISAT2 Index file. [HISAT2](#)³⁴ is a bioinformatics tool that will align the random pile of RNA reads you got through the wetlab section of the RNASeq and assign them to specific parts of the reference genome. In order to use this tool, we need to translate the genome (FASTA) and annotation (GTF) into a format that HISAT2 can read, called a “HISAT2 index file.”

```

264     index_prefix = os.path.join(genome_dir, f"{args.a}_index")
265     if not os.path.exists(f"{index_prefix}.1.ht2"):
266         build_cmd = [
267             "hisat2-build",
268             "-p", args.threads,
269             ref_fasta,
270             index_prefix
271         ]
272         run_command(build_cmd, "Building HISAT2 Index")

```

This first section of code will first create a variable named `index_prefix` that contains a subdirectory located in the `genome_dir` path that will be the accession number, followed by “`_index`.” The program will then check to make sure there is no files in that directory that end in “`.1.ht2`.” (The existence of this file would imply that this command was already run, and we don’t need to do this step. Once the computer knows that the file doesn’t exist, we prep a `build_cmd` that will be used to build the HISAT2 index in BASH, then run the same way we did in the [“Download Genome Data”](#) section.

Run FastQC

```
280 # 3A. Run FastQC on all raw reads
281
282 FastQC_Start_time = datetime.datetime.now()
283
284 print("\n" + "="*30)
285 print(f"STARTING FASTQC (time: {datetime.datetime.now().strftime(
286   ('%Y-%m-%d %H:%M:%S'))}")
287 print("="*30)
288
289 # Extract all R1 and R2 file paths from your sample_pairs list
290 all_fastq_files = []
291 for s, r1, r2 in sample_pairs:
292     all_fastq_files.extend([r1, r2])
293
294 fastqc_cmd = [
295     "fastqc",
296     "-t", args.threads, # Use the threads argument
297     "-o", fastqc_dir    # Output to the folder we just created
298 ] + all_fastq_files    # Append all the sequence files to the
299                        # command
300
301 run_command(fastqc_cmd, "Running FastQC on raw reads")
302
303 FastQC_End_time = datetime.datetime.now()
```

This line of code runs a “quality check” program called `fastqc` on each of our raw reads. This will tell us important information about how our reads went so that we know exactly how much we can trust our later analysis. Later on in our pipeline we will run another program called `multiqc` that will put all of the information in these `fastqc` files into an easy to read sheet that allows us to compare our runs.

HISAT2 Alignment Loop

```
304 # 4. Process Loop
305 print("\n" + "="*30)
306 print(f"STARTING BATCH PROCESSING (time: {datetime.datetime.now()
307       ().strftime("%Y-%m-%d %H:%M:%S")})")
308 print("="*30)
309
310 # Prepare lists for featureCounts
311 sam_list = []
312 counts_list = []
313 id_number = 1
314
315 for sample_id, r1, r2 in sample_pairs:
316     print(f"\n--> Processing Sample: {sample_id} ({id_number}
317           of {len(sample_pairs)})\nStart Time: {datetime.datetime.now()
318           ().strftime("%Y-%m-%d %H:%M:%S")}")
319
320     # Define output file paths
321     sam_output = os.path.join(aligned_dir, f"{sample_id}.sam")
322     sam_list.append(sam_output)
323
324     counts_output = os.path.join(counts_dir, f"{sample_id}
325           _counts.tsv")
326     counts_list.append(counts_output)
327
328     # A. Run HISAT2 Alignment
329
330     summary_log = os.path.join(aligned_dir, f"{sample_id}
331           _hisat2_summary.txt") # Create log path
332
333     align_cmd = [
334         "hisat2",
335         "-p", args.threads,
336         "-x", index_prefix,
337         "-1", r1,
338         "-2", r2,
339         "-S", sam_output,
340         "--summary-file", summary_log
341     ]
342     run_command(align_cmd, f"Aligning {sample_id}")
343
344     print(f"Completed {sample_id} ({id_number}/{len
345           (sample_pairs)}): {datetime.datetime.now().strftime
346           ("%Y-%m-%d %H:%M:%S")}")
347     id_number += 1
```

Now that we have an index file that we can use as a “map” for our experimental samples, we can use HISAT2 to assign each of the individual reads in our experimental FASTQs to the index file map. However, we will most likely have many different fastq files that we want to run HISAT2 on. Instead of running this program multiple times for every fastq we have, then adding them all up manually, **we can tell a program to run the same process on a list of different files in a method called batch processing.** This code does exactly this.

```
309     # Prepare lists for featureCounts
310     sam_list = []
311     counts_list = []
312     id_number = 1
```

First, these lines of code create an empty `sam_list` and `counts_list` that we can add to later. Then we assign the integer `1` to the variable `id_number` that will act as a counter so we can keep track of what the script is doing.

```
315     for sample_id, r1, r2 in sample_pairs:
316         print(f"\n--> Processing Sample: {sample_id} ({id_number}
              of {len(sample_pairs)})\nStart Time: {datetime.datetime.now
              ().strftime("%Y-%m-%d %H:%M:%S")}")
317
318         # Define output file paths
```

This block of code will loop through the `sample_pairs` list we made and let the user know what files the computer is working on. If you are confused about the syntax of the `for` loop see the [“Verify Fastq Files”](#) section.

```
320         sam_output = os.path.join(align_dir, f"{sample_id}.sam")
321         sam_list.append(sam_output)
322
323         counts_output = os.path.join(counts_dir, f"{sample_id}
324         _counts.tsv")
        counts_list.append(counts_output)
```

This next block of code creates output file names for the two expected outputs for this program (SAM files and a tab-separated values sheet containing counts). The program will take the `sample_id` that it's currently looking at and make a file named `<sample_id>.sam` or `<sample_id>_counts.tsv` in either an `align_dir` or `counts_dir` subdirectory, respectively. The program will then add that specific file path to our previously created `sam_output` or `counts_output` list.

```
330         align_cmd = [
331             "hisat2",
332             "-p", args.threads,
333             "-x", index_prefix,
334             "-1", r1,
335             "-2", r2,
336             "-S", sam_output,
337             "--summary-file", summary_log
338         ]
```

This next block of code prepares an `align_cmd` that we want to run on each of our `sample_id`'s. Much like the code in the "[Download Genome Data](#)," the command will enter the parts in " " exactly, the variables outside of the quotations will enter their values. Each comma separation will represent a space entered into the command line. For more info on what each of the command parameters does, enter `hisat2 -help`.

FeatureCounts

```
356     base_count_cmd = [  
357         "featureCounts",  
358         "-T", args.threads,  
359         "-p",          # Paired-end  
360         "--countReadPairs", # Count fragments instead of reads ---  
361         # commented out for testing  
362         # "-t", "exon",      # Feature type --- Commented out for  
363         # testing  
364         # "-g", "gene_id",   # Attribute type --- Commented out for  
365         # testing  
366         "-a", ref_gtf,  
367         "-o", "rnaseq_output/counts/alignedReads.tsv", # Output file  
368     ]  
369  
370     count_cmd = base_count_cmd + sam_list  
371  
372     run_command(count_cmd, f"Counting {len(sam_list)} samples with  
373     featureCounts")  
374  
375     fc_end_time = datetime.datetime.now()  
376     print(f"featureCounts End Time: {datetime.datetime.now().strftime  
377     ('%Y-%m-%d %H:%M:%S')}")
```

This next block of code sets up the computer to run featureCounts, a tool to physically determine which gene each contig maps to. This code works similarly to the

other commands that we have used with one major exception. Towards the bottom we make a new variable that combines the `base_count_cmd` that we just set up with the `sam_list` that holds the directory of all the sam files. This appends the directories of all the .sam files to the end of the command which tells the computer to run the `base_count_cmd` on all of the listed .sam files. The `run_cmd` function executes our command in BASH.

Quality Control

```
380 qc_cmd = [  
381     "multiqc",  
382     "-o", multiqc_dir,  
383     "."  
384 ]
```

Our final functional command runs a program called MultiQC. This tool makes an easy-to-read Quality Control document that gives you an idea of how reliable the data is.

R Analysis - What do the numbers mean?

The next part of the analysis consists of automated R-analysis. The majority of the analysis is done using a package called `rpy2`. This acts as a bridge between R and Python and allows us to automate the R analysis process.

```

391     # Do R analysis
392     r_start_time = datetime.datetime.now()
393
394     print("\n" + "="*30)
395     print("Starting R Analysis")
396     print("="*30 + "\n")
397
398     base = importr('base')
399     deseq = importr('DESeq2')
400     pheatmap = importr('pheatmap')
401     tidyverse = importr('tidyverse')
402     deseq2 = importr('DESeq2')
403     ggplot2 = importr('ggplot2')
404     pheatmap = importr('pheatmap')
405     dplyr = importr('dplyr')
406     fgsea = importr('fgsea')
407     msigdb = importr('msigdb')
408     lazyeval = importr('lazyeval')
409     EnhancedVolcano = importr('EnhancedVolcano')
410
411     counts_file = "rnaseq_output/counts/alignedReads.tsv"
412     metadata_file = "inputFiles/metadata.csv"
413
414     rDiag_dir = "rnaseq_output/rFigures/diagnostics"
415     os.makedirs(rDiag_dir, exist_ok=True)
416
417     # 1. Read the metadata to identify all unique conditions
418     metadata = pd.read_csv(metadata_file)
419
420
421
422     unique_conditions = metadata['Condition'].unique()
423
424     # 2. Define your control group and extract the experimental
425     # groups
426     control_group = "Control"
427     experimental_groups = [cond for cond in unique_conditions if
428                            cond != control_group]

```

This first block of code imports the different R packages we are going to need for our R analysis. To shorten the overall length of this colossal document, see the documentation

for each of the packages by entering `?<package name>` into the console of R-Studio for each of the following packages. The next two lines set variables for the featureCounts reads and metadata file. The next two lines create a directory for the diagnostic plots that we will be making later on in the script. Finally the last lines define the experimental and control groups based on the metadata.

Data Prep

```
432 r_dataPrep = rf""
433 library(DESeq2)
434
435 # 1. Read the featureCounts file
436 raw_counts <- read.table("{counts_file}", header = TRUE, sep =
"\t", stringsAsFactors = FALSE, skip = 1)
437
438 # 2. Extract the count matrix (columns 7 to the end)
439 count_matrix <- raw_counts[, 7:ncol(raw_counts)]
440 rownames(count_matrix) <- raw_counts$Geneid
441
442 # 3. Clean up the sample names in the column headers
443 clean_sample_names <- colnames(count_matrix)
444 clean_sample_names <- gsub("^.*\.\alignments\\.\"", "",
clean_sample_names)
445 clean_sample_names <- gsub("\\.sam$", "",
clean_sample_names)
446 colnames(count_matrix) <- clean_sample_names
447
448 # 4. Load the real metadata table directly
449 metadata <- read.csv("{metadata_file}", stringsAsFactors = FALSE)
450 rownames(metadata) <- metadata$SampleID
451 metadata <- metadata[colnames(count_matrix), ] # Ensure matching
order
452 metadata$Condition <- as.factor(metadata$Condition)
453
454 # 5. Create DESeq2 Object
455 dds <- DESeqDataSetFromMatrix(countData = count_matrix, colData
= metadata, design = ~ Condition)
456
457 # Run DESeq to calculate dispersions
458 dds <- DESeq(dds)
459
460 # 6. Variance Stabilizing Transformation
461 vsd <- vst(dds, blind = FALSE)
462 ""
463
464 # Execute the code in R
465 robjects.r(r_dataPrep)
466
467 print("data prepped...")
```

This code runs six different steps that prep the featureCounts table that was output by the first part of our Python script into a format that is usable by R. **Please note that each of the R code chunks will be formatted as an R chunk saved as a variable, then executed using `objects.r(<code chunk variable>)`. This is the formatting for running `rpy2` in Python.** Here are the specifics for each step:

1. This step reads the featureCounts table in our directory, tells the program to `skip` the first line (which consists of run metadata), reads the first row as a `header`, and treats each tab (notated as `\t`) as a separation, then saves it as the variable `r_counts`.
2. This step extracts the table columns that contain the counts we want to a variable called `count_matrix`. The first 7 columns are metadata and can be disregarded for this analysis.
3. This step cleans up the headers in the `count_matrix`. The read names are defined as their full file path. This code uses the R `gsub` package to remove the filepath and the suffix so that only the read ID remains. Then those clean column names are put back into our `count_matrix` variable to make the analysis easier.
4. This step loads our uploaded metadata file so that R can see it. The program reads the directory that we listed in our `metadata_file` variable, then ensures the file is in a format the program can read.
5. This step uses an RNAseq analysis program called DESeq2 to analyze the differential expression results from our featureCounts table. The program pulls

data from the previously made `count_matrix` file and separates the different experimental groups we defined in our metadata file.

6. Applied a variance stabilizing transformation to the data to account for any multimodal distribution of data. In essence, this allows for our data to be analyzed even if the counts don't follow a normal distribution.

Analysis Loop - Same Analysis, Different Experimental Groups

The next section of code is a `for` loop that will automatically compare each of our experimental groups listed in our metadata with our control group. For each comparison, a table showing raw differentially expressed values is created, as well as a GSEA and Volcano Plot. Most of the code works similarly to previous code, so to save time, I will refrain from going step by step through each line of code and will instead highlight some of the more interesting or complex code. For more information, see the comments within the code itself.

Graphing Principal Component Analysis

```
473   r_pca = rf"""
474   plotPCA(vsd, intgroup = "Condition") +
475   theme_minimal() +
476   ggtitle("PCA of RNA-seq Samples") +
477   geom_text(aes(label=name), size=3)
478
479   ggsave("pcaGraph.png", path = "rnaseq_output/rFigures/
480   diagnostics")
481   """
482   # Execute the code in R
483   robjects.r(r_pca)
484
485   print("pca graphed...")
```

The above code block creates a principal component analysis (PCA) plot using our analyzed data. Please refer to the [Quality Control Check](#) section and [Figure 3.15](#) for more details.

Making a Dispersion Estimates Graph

```
489     r_dispersion = rf"""
490
491
492     # 1. Open a PNG device and specify the full path and filename
493     png("rnaseq_output/rFigures/diagnostics/dispersionEstimates.
494         png", width=800, height=600)
495
496     # 2. Draw the base R plot (it routes directly to the PNG,
497         skipping the pop-up)
498     plotDispEsts(dds, main="Dispersion Estimates")
499
500     # 3. Close the device to finalize and save the file
501     dev.off()
502     """
503
504     # Execute the code in R
505     robjects.r(r_dispersion)
506
507     print("dispersion graph made...")
```

The above code block creates a dispersion estimate plot using our data. Please refer to the [Dispersion Plot](#) section and [Figure 3.20](#) for more details.

Extracting Target Species

```
518 # 1. Automatically extract the species from the NCBI assembly
    report
519 jsonl_path = 'rnaseq_output/genome_data/ncbi_dataset/data/
    assembly_data_report.jsonl' # Update with your path
520 with open(jsonl_path, 'r') as f:
521     # A .jsonl file contains one JSON object per line. We just
    need the first line.
522     assembly_data = json.loads(f.readline())
523
524 # Extract the species name (e.g., "Canis lupus familiaris")
525 target_species = assembly_data['organism']['organismName']
526 print(f"Target species extracted from assembly report:
    {target_species}")
527
528 # 2. Read the metadata to identify conditions (No need to change
    your metadata.csv!)
529
530
531 # Optional: Validate the extracted species against msigdb to
    catch spelling mismatches early
532 try:
533     robjects.r(f"""
534     library(msigdb)
535     valid_species <- msigdb_species()$species_name
536     if (!("{target_species}" %in% valid_species)) {{
537         stop("Error: '{target_species}' is not a valid species
        in msigdb.")
538     }}
539     """)
540     print("Species validation passed!")
541 except Exception as e:
542     print(f"Species validation failed: {e}")
543     exit(1)
```

In order to make a GSEA later on in the code, we need to use the `msigdb` package to extract the relevant metabolic pathways for our reference organism. In R, we would type out the species name (*Canis lupus familiaris*) into R. This causes a problem when trying to automate the analysis in the simplest way possible because we

may want to use this pipeline for RNAseq on an organism that is not a dog. While we could add another argument in the [“Creating Arguments”](#) section, this adds more points of failure (misspellings, forgotten input, etc.) and unnecessary complexity to a pipeline that is supposed to be as simple as possible.

To solve this problem, **this block of code looks into an Assembly Data Report that is automatically downloaded alongside our FASTA and annotation and pulls the organism name from the metadata.** This file is always downloaded when you pull information from the NCBI database and contains structured information on how the data was collected. This code will pull the species name from this file and automatically insert it into the R code in the structure that it wants.

Making a Dispersion Estimates Table

```
578 r_difExp = f""
579 # 1. Extract the results for your specific comparison
580 res <- results(dds, contrast=c("Condition", "{exp_group}", "{control_group}"))
581
582 # 2. Order the results by the adjusted p-value
583 res <- res[order(res$padj), ]
584
585 # 3. Filter for significance (FDR < 0.05) and effect size
586     (absolute log2FC > 1)
587 sig_res <- subset(na.omit(res), padj < 0.05 & abs
588     (log2FoldChange) > 1)
589
590 # 4. Save the significant genes to a CSV file for your
591     records
592 write.csv(as.data.frame(sig_res), file="rnaseq_output/
593     rFigures/results/{exp_group}/deg/Significant_DEGs_{exp_group}
594     _vs_{control_group}.csv")
595 ""
596
597 robjects.r(r_difExp)
598 print(f"Completed DEGs for {exp_group} vs {control_group}!")
```

This chunk of code will create a table of differentially expressed genes that lists information on the specific genes that were over- and underexpressed. The table will be outputted as a “.csv” file in the results folder of this pipeline. See the [Differentially Expressed Genes Table](#) section or [Figure 3.23](#) for more information.

Making a Volcano Plot

```
601 r_volcano = f""
602 top5 <- head(rownames(subset(na.omit(as.data.frame(res)), abs
    (log2FoldChange) > 1 & padj < 0.05)), 5)
603
604 # Generate the plot
605 EnhancedVolcano(res,
606   lab = rownames(res),
607   selectLab = top5,          # <-- only label these genes
608   x = 'log2FoldChange',
609   y = 'padj',
610   title = '{exp_group} vs {control_group}',
611   legendLabels = c("Not Influential & Insignificant",
    "Influential", "Significant", "Influential &
    Significant"),
612   caption = NULL,
613   pCutoff = 0.05,
614   FCcutoff = 1,
615   pointSize = 2.0,
616   labSize = 3.0,
617   drawConnectors = TRUE,    # draws lines from label to
    point
618   widthConnectors = 0.5,
619   arrowheads = FALSE,
620   min.segment.length = .1,
621   col = c('grey30', 'forestgreen', 'royalblue', 'red2'),
622   legendPosition = 'right')
623
624 ggsave("volcanoPlot-{exp_group}.png", path = "rnaseq_output/
    rFigures/results/{exp_group}/volcanoPlot")
625 ""
626
627 robjects.r(r_volcano)
628 print(f"Completed volcano plot for {exp_group} vs
    {control_group}!")
```

This chunk of code will create a volcano plot that shows the distribution of gene influence and significance. For more information on the volcano plot, see the [Volcano Plot](#) section and [Figure 3.24](#).

Making a GSEA Plot

```
636 r_gsea = f""
637 library(fgsea)
638 library(dplyr)
639 library(ggplot2)
640 library(msigdb)
641
642 # =====
643 # 1. Prepare the Ranked Gene List
644 # =====
645 res_clean <- res[!is.na(res$stat), ]
646 ranks <- res_clean$stat
647 ranks <- ranks + rnorm(length(ranks), sd=1e-10) # Jitter to
        prevent ties
648 names(ranks) <- rownames(res_clean)
649 ranks <- sort(ranks, decreasing = TRUE)
650
651 # =====
652 # 2. Load Gene Sets (Pathways) using msigdb
653 # =====
654 pathways_df <- msigdb(species = "{target_species}",
        collection = "H")
655 pathways <- split(x = pathways_df$gene_symbol, f =
        pathways_df$gs_name)
656
657 # =====
658 # 3. Run fgsea
659 # =====
660 fgseaRes <- fgsea(pathways = pathways,
661                  stats = ranks,
662                  minSize = 15,
663                  maxSize = 500)
664
665 fgseaRes <- fgseaRes[order(fgseaRes$NES, decreasing =
        TRUE), ]
666
667 # Flatten the leadingEdge list column to avoid the
        'EncodeElement' CSV error
668 fgseaRes_save <- fgseaRes
669 fgseaRes_save$leadingEdge <- sapply
        (fgseaRes_save$leadingEdge, paste, collapse = ";")
670
671 write.csv(as.data.frame(fgseaRes_save),
672          file="rnaseq_output/rFigures/results/{exp_group}/
        gsea/GSEA_{exp_group}_vs_{control_group}2.csv",
673          row.names=FALSE)
```

```

677 # =====
678 # 4. Visualize and Save the Results
679 # =====
680
681 if(nrow(fgseaRes[ES > 0]) > 0) {{
682     topPathwaysUp <- fgseaRes[ES > 0][head(order(pval),
683     n=1), pathway]
684     topPathwayRes <- fgseaRes[pathway == topPathwaysUp, ]
685
686     p <- plotEnrichment(pathways[[topPathwaysUp]], ranks) +
687     labs(
688         title = paste("Enrichment Plot:", topPathwaysUp),
689         subtitle = paste0(
690             "p-value = ", signif(topPathwayRes$pval, 3),
691             " | padj = ", signif(topPathwayRes$padj,
692             3),
693             " | NES = ", round(topPathwayRes$NES, 2)
694         ) +
695         theme_minimal()
696
697     ggsave(filename="GSEA_{exp_group}_vs_{control_group}
698     2.png", plot=p,
699     path="rnaseq_output/rFigures/results/{exp_group}/
700     gsea",
701     width=8, height=6, bg="white")
702 }} else {{
703     print("No pathways had an Enrichment Score > 0 to plot.")
704 }}
705
706 ""
707
708 robjects.r(r_gsea)
709 print(f"Completed GSEA plot for {exp_group} vs
710 {control_group}!")
711
712 print(f"{exp_group} analysis complete...")
713
714 r_end_time = datetime.datetime.now()
715
716 print(f"\n{(' '*30)}\nR Analysis Complete\n Total time:
717 {r_end_time - r_start_time}\n {(' '*30)}\n ")

```

This final chunk of code creates a GSEA plot and is divided into four steps. The first step prepares a gene list ranked from the most overexpressed to the most underexpressed for the GSEA plot to refer to. The second uses a package called `msigdbR` to pull metabolic pathway information on the `target species` we pulled from the JSON file. The third step runs the GSEA algorithm on the differentially expressed genes to find how the genes fit into the overall metabolic pathways. The fourth and final step visualizes the best fitting GSEA into a plot. For more information on how to read the GSEA plot see the [GSEA Plot](#) section and refer to [Figure 3.24](#).

Summary

```
727 print("="*30)
728 print(f"Summary of Pipeline\n")
729 print(f"Total Time:\n {script_start_time - script_end_time}\n
({script_start_time.strftime("%Y-%m-%d %H:%M:%S")} -
{script_end_time.strftime("%Y-%m-%d %H:%M:%S")})\n")
730 print(f"Pre-flight Check Time:\n {preflight_start_time -
preflight_end_time}\n ({preflight_start_time.strftime("%Y-%m-%d
%H:%M:%S")} - {preflight_end_time.strftime("%Y-%m-%d %H:%M:%S")})
\n")
731 print(f"FASTQ Verification Time:\n {ver_fastq_start_time -
ver_fastq_end_time}\n ({ver_fastq_start_time.strftime("%Y-%m-%d
%H:%M:%S")} - {ver_fastq_end_time.strftime("%Y-%m-%d %H:%M:%S")})
\n")
732 print(f"Directory Setup Time:\n {setup_dir_start_time -
setup_dir_end_time}\n ({setup_dir_start_time.strftime("%Y-%m-%d
%H:%M:%S")} - {setup_dir_end_time.strftime("%Y-%m-%d %H:%M:%S")})
\n")
733 print(f"Genome Download Time:\n {genome_download_start_time -
genome_download_end_time}\n ({genome_download_start_time.strftime
("%Y-%m-%d %H:%M:%S")} - {genome_download_end_time.strftime
("%Y-%m-%d %H:%M:%S")})\n")
734 print(f"Locate Genome Files Time:\n
{locate_genome_files_start_time - locate_genome_files_end_time}
\n ({locate_genome_files_start_time.strftime("%Y-%m-%d
%H:%M:%S")} - {locate_genome_files_end_time.strftime("%Y-%m-%d
%H:%M:%S")})\n")
735
736 print(f"FastQC Start Time:\n {FastQC_Start_time -
FastQC_End_time}\n ({FastQC_Start_time.strftime("%Y-%m-%d
%H:%M:%S")} - {FastQC_End_time.strftime("%Y-%m-%d %H:%M:%S")})
\n")
737
738 print(f"HISAT2 Index Build Time:\n
{hisat2_index_build_start_time - hisat2_index_build_end_time}\n
({hisat2_index_build_start_time.strftime("%Y-%m-%d %H:%M:%S")} -
{hisat2_index_build_end_time.strftime("%Y-%m-%d %H:%M:%S")})\n")
739 print(f"featureCounts Time:\n {fc_start_time - fc_end_time}\n
({fc_start_time.strftime("%Y-%m-%d %H:%M:%S")} - {fc_end_time.
strftime("%Y-%m-%d %H:%M:%S")})\n")
740 print(f"MultiQC Time:\n {qc_start_time - qc_end_time}\n
({qc_start_time.strftime("%Y-%m-%d %H:%M:%S")} - {qc_end_time.
strftime("%Y-%m-%d %H:%M:%S")})\n")
741 print(f"R Analysis Time:\n {r_start_time - r_end_time}\n
({r_start_time.strftime("%Y-%m-%d %H:%M:%S")} - {r_end_time.
strftime("%Y-%m-%d %H:%M:%S")})\n")
742 print("="*30)
```

This final chunk of code displays a final report on how long it took to run each section. This should be used for reference, but doesn't serve any functional purpose in the pipeline.

Chapter 3 - Analysis

We got oriented with the tools we needed in part 1, and we walked through an example in part 2. Now we need to turn the cryptic scramble of genes and numbers that we got into our output into meaningful results that can be communicated with other scientists through analysis. This is arguably the most important part of a bioinformaticist's job because most of the output of bioinformatics tools is unintelligible to scientists who don't have hours to sift through mountains of data to find the most important. Therefore, as the scientist that is closest to the data, it is your job to effectively communicate what the results are, what it means, and – most importantly – how confident we are in our findings. **In this section, we will use the output from the previous part and translate it into meaningful results using R.**

One final important note before we move on. Most people like to think of statistics as completely objective. However, there is also an art to statistical analysis in how you “manipulate” the data. Data manipulation is not necessarily a bad thing; almost all data needs to be manipulated in some way to find meaningful results. For example, without any data manipulation, if you wanted to estimate the wealth of the everyday US citizen, you would find that the average net worth of \$1,063,700.³⁶ The everyday US citizen isn't a millionaire, but there are a few outliers who make absurd amounts of money that skew the average. However, once you account for these outliers by using the median, you find that it's closer to \$192,900.

Depending on your “goal,” you can manipulate data to fit a motive. However, as a scientist, you have a responsibility to report data as objectively as possible. It may be tempting at times to manipulate data to make it more impactful, especially when there's financial or academic pressure to find something impactful. However, this kind of data manipulation can leave a major stain on your scientific career. Always report how you manipulate your data and be truthful about what your data can be applied to and what it can't.

Quality Control Check

There's a common saying that is especially important with regard to bioinformatics and science as a whole: “Garbage in, garbage out.” This means that in order for us to put any stock into our results, we need to determine if the data we are working with is acceptable. **This is called quality control or QC.** There are many different methods of going about it. While there is no definite “stop point,” it is best practice to run multiple QCs to find the strengths and weaknesses of your data.

MultiQC

In our pipeline, we included software called FastQC and MultiQC that turns all of the useful checks we need into an easy-to-read HTML file. The MultiQC HTML file will consist of all of the FastQC information together. The top of the MultiQC report contains

a summary of some of the more important diagnostic information in the format shown above. More detailed information on each of the statistics can be found below the summary, and we will go into detail on what to look for below.

Figure 3.1: MultiQC - General Statistics Overview

Sample Name ^	Assigned	% Aligned	Dups	GC	Seqs
ERR1899753	71.0%				
ERR1899753_1			54.6%	49.0%	14.3 M
ERR1899753_2			49.2%	50.0%	14.3 M
ERR1899753_hisat2		97.1%			

Figure 3.1. A screenshot of the general statistics table that is outputted at the beginning of the MultiQC HTML Document.

Gene Assignments - Mapping to Genes

Gene assignment graphs measure how many of our reads are mapped to specific genes. We hope to find a high proportion of assigned reads and a high sequencing depth (reads mapped).

Figure 3.2 Ideal Feature Counts

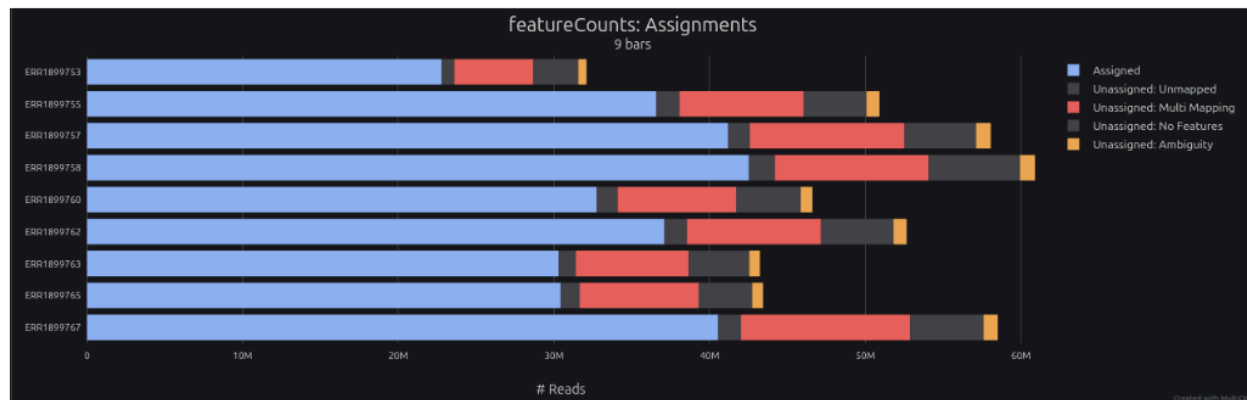


Figure 3.2. Figure 3.2: An histogram showing an ideal featureCounts assignment output showing the read assignment distribution for nine (9) paired end sequencing libraries. Blue bars show the number of reads in each library that were uniquely and appropriately assigned to the reference genome. The first grey bar shows the amount of reads that were not able to be mapped to the reference genome (improved with better sequencing quality). Red bars represent sequences that mapped to multiple parts of the reference genome (likely caused by repetitive genomic sequences). The second grey bar represents areas that mapped to unannotated parts of the genome (possibly due to incomplete genome annotation files). Finally the orange bar represents reads mapped to areas that overlap multiple different gene annotations (often caused by densely annotated genome sections).

Sequence Counts - Unique Mapping

The sequence counts graph displays how many of our reads map to a unique part of the genome. We expect to see some repeats, but want to make sure that we have enough unique reads to cover a good portion of the genome.

Figure 3.3: Ideal FastQC Sequence Counts Histogram

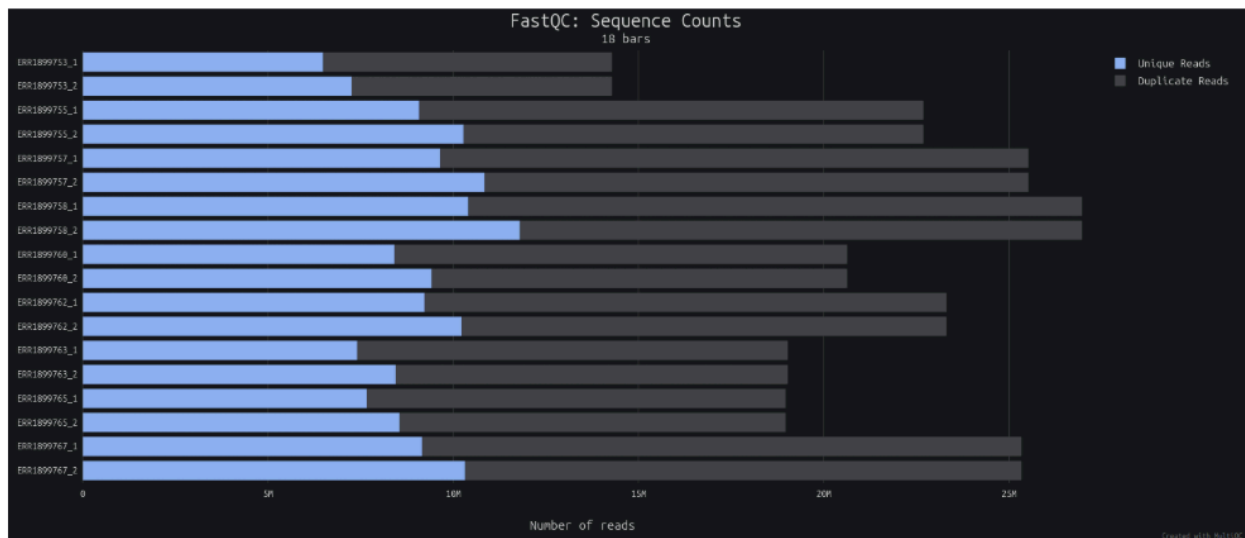


Figure 3.3: A histogram showing the number of unique and duplicated reads found in nine (9) paired-end sequencing libraries. Blue bars show the number of reads that have a unique nucleotide sequence among all reads from a single library. Grey bars represent reads in which nucleotide sequences are present more than once in a single library. Minimum read amounts will be highly dependent on the model organism and the tool used.⁵⁰

Figure 3.4: Poor FastQC Sequence Counts Histogram

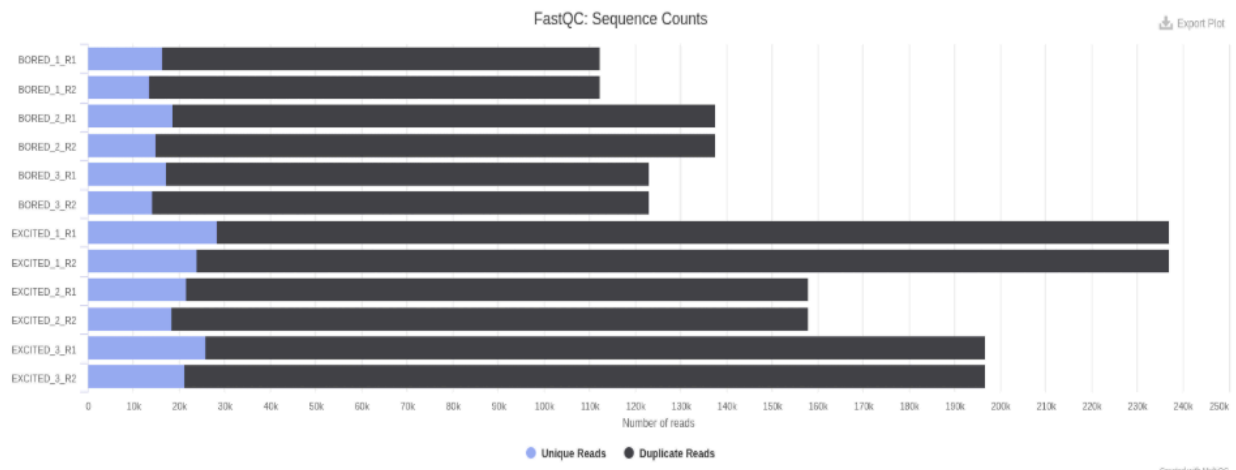


Figure 3.4: An example of a FastQC sequence counts histogram that would not be likely to produce reliable data. Bars aligned on the Y-axis each represent a sample's forward (R1) or reverse (R2) read, while the length of the bar represents the total number of reads (sequencing depth). The total number of unique reads is represented in blue, and duplicate reads in grey. Each sample has fewer than 250k total reads and contains a very small proportion of unique reads, which could contribute to poor-quality data. (MultiQC, 2016).⁴³

Sequence Quality Histogram- How Good Are Our Reads as bp Increases?

The sequence quality histogram shows the “Phred scores” of our reads. In short a Phred score is an indication of how confident we are that a specific nucleotide is what we say it is, as measured in [Equation 3.1](#). In short, the larger the Phred score, the more confident you are that nucleotide you’re looking at is accurate.

Equation 3.1: Phred Score Equation

$$Q = -10 \log_{10} P$$

P = Error Probability

Q = Phred Score

Equation 3.1. The equation used to calculate the Phred score of a nucleotide. In short, a Phred score can be thought of as having a “one in Q” chance of being wrong.

Figure 3.5: FastQC Mean Quality Score Plot (Ideal Example)



Figure 3.5: An ideal example of the mean quality scores line graph. to assess the quality scores of your reads in each of your samples. The Phred quality score at each base pair position in your sample is plotted on a line graph. For reliable results, you ideally want to see a stable horizontal line that stays in the green zone, especially towards longer reads.

Figure 3.6: FastQC Mean Quality Score Plot (Poor Example)

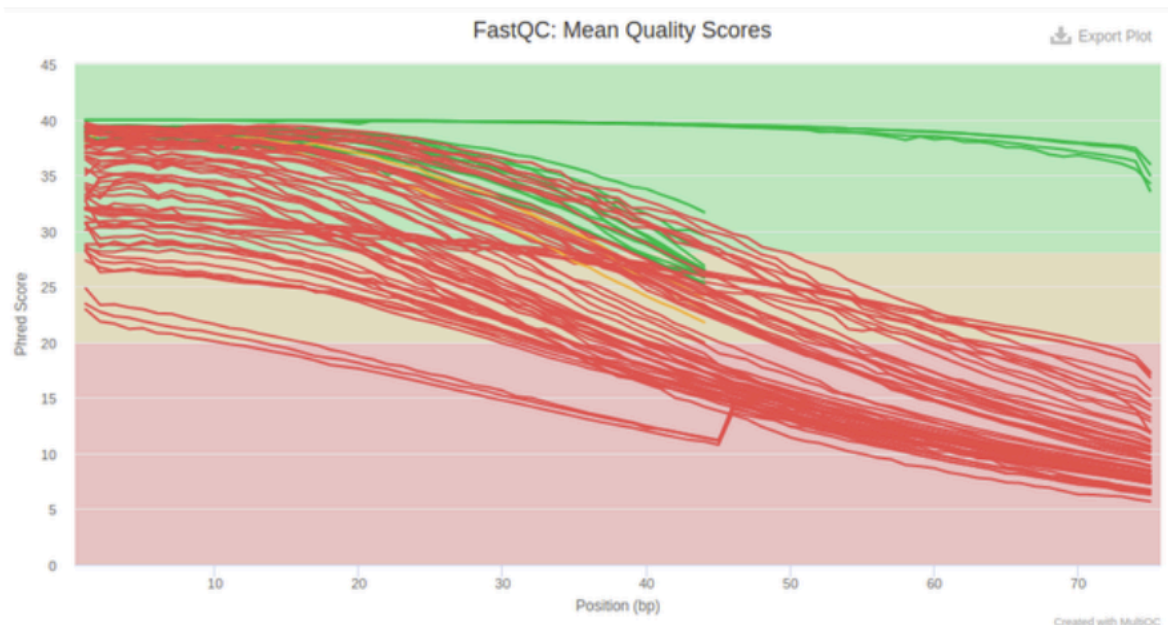


Figure 3.6. A poor example of a mean quality score plot. Each line represents a sample that had its mean Phred quality score plotted for every base pair in the length of the sample. The most important indication that this data is unreliable is that the quality score dips under what is considered “good” (green, Phred = 27), especially during the second half of the sequencing. Unfortunately, there is no way to fix this problem outside of rerunning the experiment (Biostar Forum, 2020).⁴⁵

Per Sequence Quality Scores - How Good Are Our Samples on Average

The per sequence quality score graph is similar to the sequence quality histogram except that it displays the average quality across the entire read. We are looking for a peak in the green area.

Figure 3.7: FastQC Per Sequence Quality Score (Ideal Example)



Figure 3.7. An ideal per-sequence quality score plot. The graph maps the number of reads that were measured for each of the Phred quality scores in each sample and connects them in a line. This plot is ideal because the majority of our reads fall in the “good” region (green, Phred > 27). The high quality of reads gives us confidence in our later results.

Figure 3.8: FastQC Per-Sequence Quality Score (Poor Example)



Figure 3.8. A poor quality example of a per-sequence quality score graph. The graph shows a line graph of the total number of reads that were counted per Phred score. What makes this low-quality graph is that the majority of our reads in this sample fall in the “poor quality” region (red, Phred < 20). The low average quality score of this sample lowers the confidence that we can trust later findings. Unfortunately, there is no fix for these scores short of rerunning the experiment.

Per Base Sequence Content - Unusual Sequence Content

The per-base sequence content shows a color-coded figure that displays a section of base pairs and the average nucleotides that make up that section. We may see an abnormally high percentage of a specific bp towards the beginning due to a known bias in the library prep process (especially with Illumina sequenced samples⁴⁷). However, we should see a relatively equal proportion of nucleotide content with complementary reads having complementary majority base pairs (for example, the latter end of ERR1899762_1 has a red hue indicating that it's mostly Thymine, while ERR1899762_2 has a slightly green hue indicating that it's mostly Adanine).

The only exception is that occasionally some organisms – like thermophiles⁴⁷ – will have a higher than average GC content. If you suspect there's an unusually high amount of guanine and cytosine, check to see if the model organism you are working on is expected to have a higher than average GC content.

Figure 3.9: Per Base Sequence Quality Heatmap Example

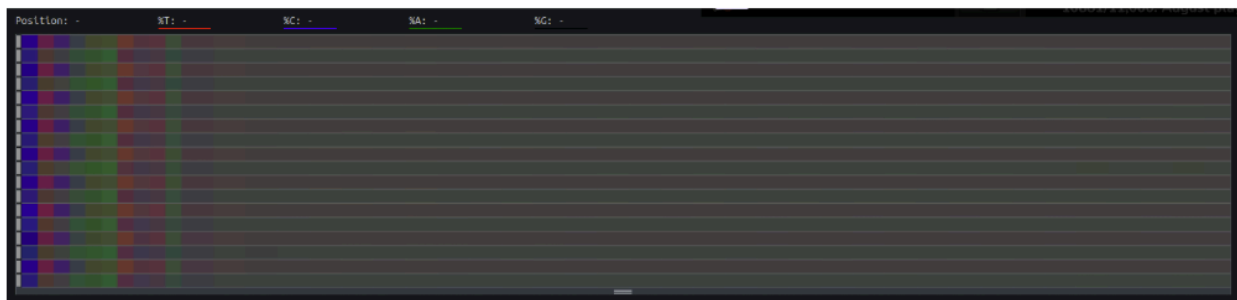


Figure 3.9. An example of a per-base sequence quality heatmap. The heatmap displays the most common nucleotide represented at each base pair across a sample as a color (A = Green; T = Red; G = Grey; C = Blue). In an ideal sample like this one, you should expect the forward and reverse reads to have a majority of complementary bases, such as the alternating red and green stripes. There is also a strong bias (more vibrant colors) towards the beginning of the samples. This is not unusual for NGS sequencing and represents primers utilized in the sequencing process. They should not affect our analysis.

Per Sequence GC Content - Identify Nucleotide Composition

The per sequence GC content is a graph that checks to see what the average Guanine-Cytosine concentration is in your samples. Ideally, you should see a normal distribution of GC content.

Figure 3.10: Per Sequence GC Content Graph

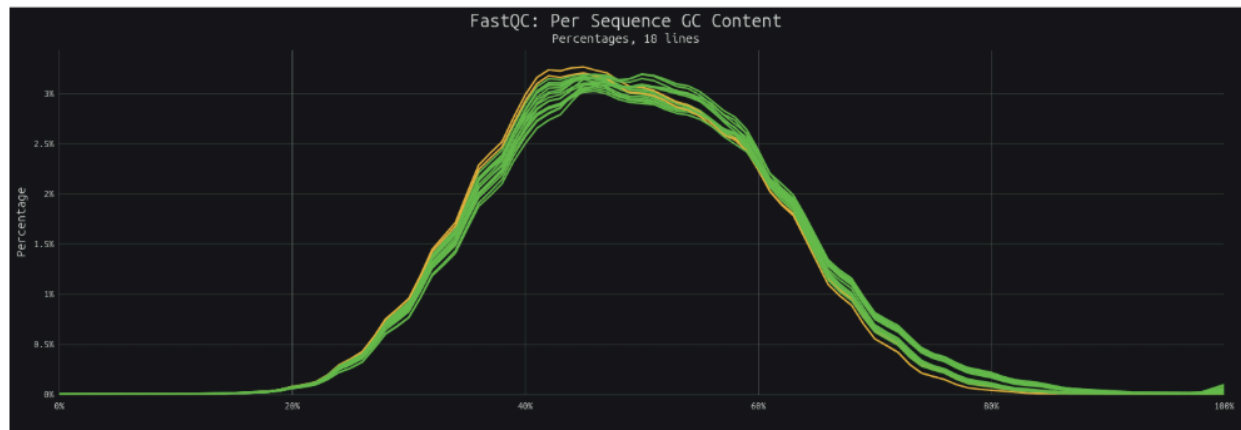


Figure 3.10. An ideal per-sequence GC graph. The graph shows the percentage of Guanine and Cytosine base pairs in each read of the sample. In an ideal sample like this, we should see a normal distribution form with most samples overlapping. The average percentage of the normal distribution forms around is species-dependent. Quality results like this give us confidence in our downstream analysis.

Figure 3.11: FastQC Per-Sequence GC Content (Poor Example)

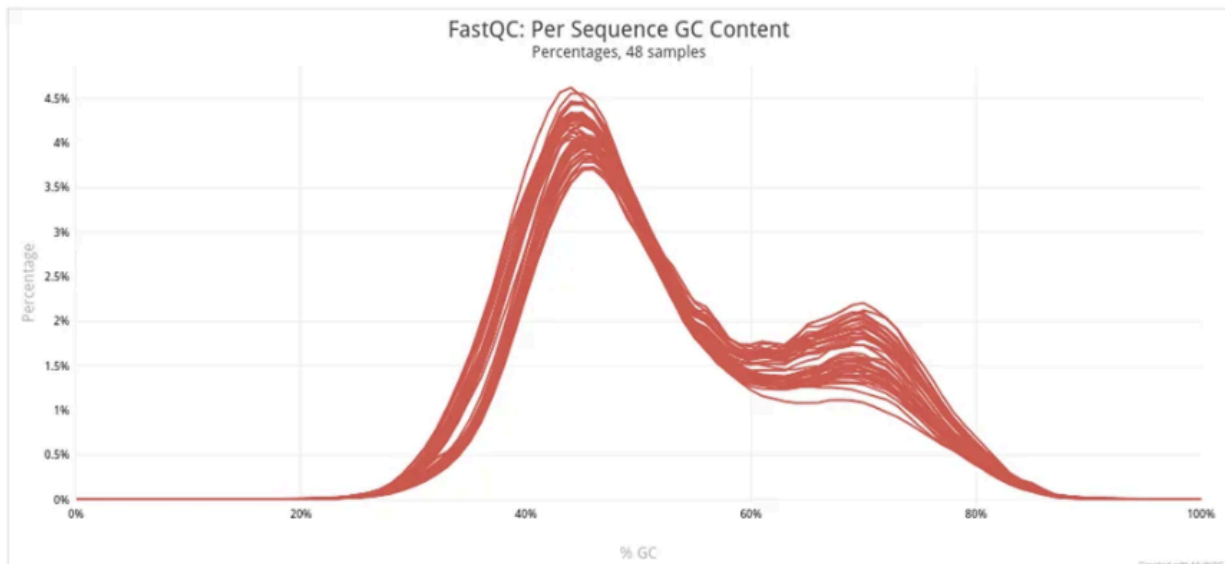


Figure 3.11. A poor example of a per-sequence GC content graph. The main concern of this graph is its bimodal distribution of peaks at around 45% and 70%. A bimodal distribution like this is often due to some sort of contamination by an organism with a different GC content than the target. The most common contaminant is the host organism's rRNA, but contaminant species can be identified by feeding overrepresented samples into NCBI's BLAST. Once the species is identified, decontamination programs can remove contamination.⁴⁸

Adapter Content - Were Adaptors Removed?

The last important graph we need to look at is the adaptor content graph. Typically, bioinformatic programs will remove the adaptors that are added to the target DNA. These adaptors will be known nucleotide sequences that are specific to the sequencer used. We should see a flat graph that doesn't show any peaks.

Figure 3.12: FastQC Adaptor Content Graph (Ideal)

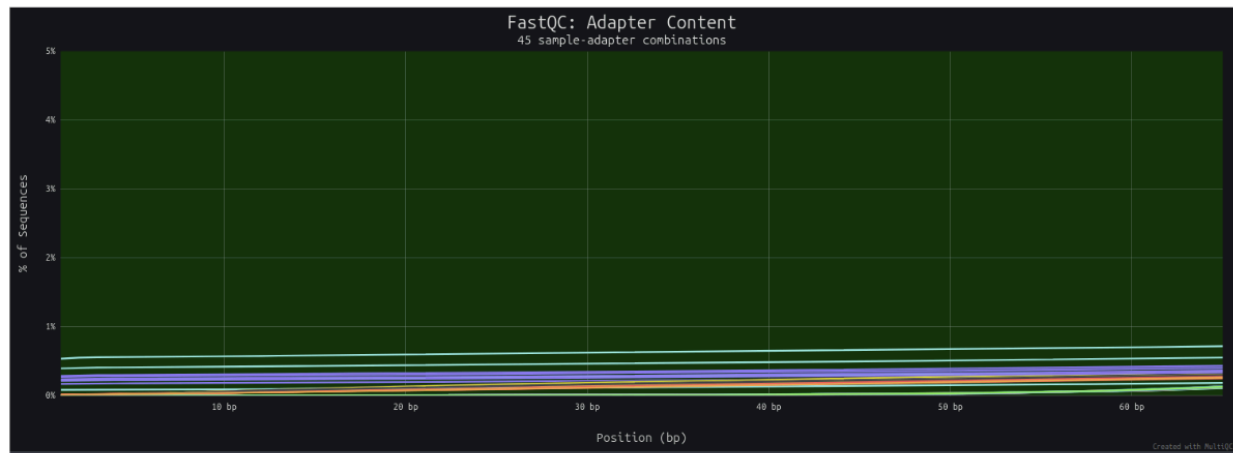


Figure 3.12. An ideal example of an adaptor content graph. The graph shows the percentage of reads that map to known adaptor sequences at each base pair. An ideal graph will have an extremely low percentage of adaptor sequences with no peaks. Adaptors present in our samples would skew our results.

Figure 3.13: FastQC Adaptor Content Graph (Poor Example)

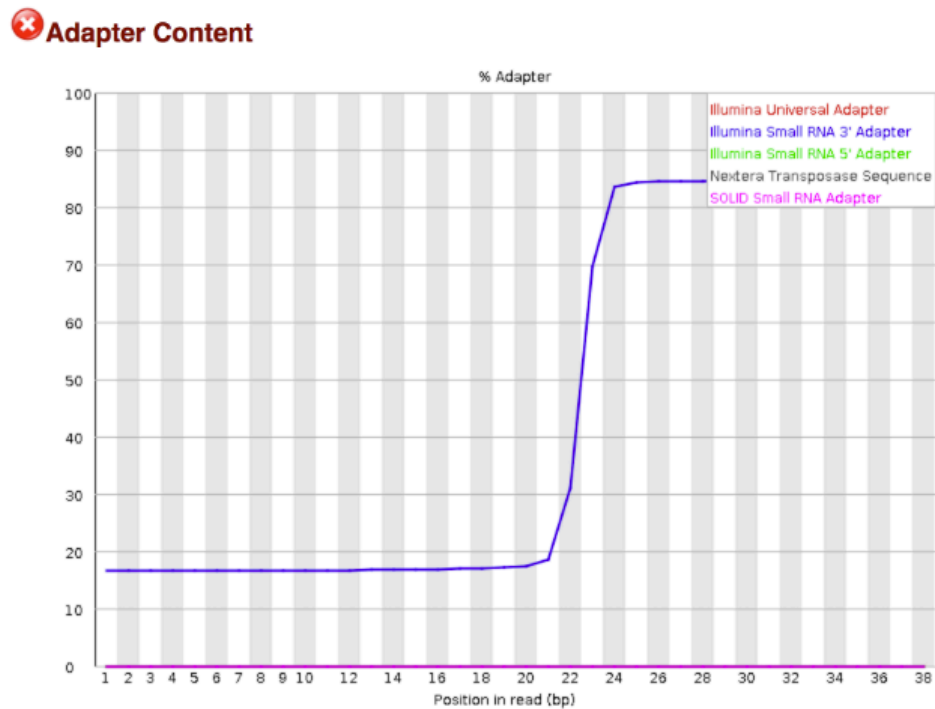


Figure 3.13. An example of a poor adaptor content graph. The graph is problematic as one sample has a spike in Illumina Small 3' Adaptor percentage around the 22bp area. Adaptors like the one shown in the graph are supposed to be removed during the pipeline, but can sometimes sneak through and disrupt downstream analysis. Adaptors like this one can be removed using specialized software.

FastQC Status Check

The final heatmap at the bottom shows a status check of all the previous graphs. The only other thing of note is the red and yellow lines under “per base sequence

content” and “sequence duplication levels.” This is actually completely normal for RNA-seq and is caused by library prep bias. The rest of our checks are alright.

Figure 3.14: FastQC Status Check Example



Figure 3.14. An example of the final status check heat map. This heatmap displays how each of your samples (Y-axis) performed in each of FastQC's tests as either green (good), yellow (suspect), or red (needs attention). While you should recheck any red or yellow tests, red and yellow bands that are consistent across all samples aren't an immediate cause for concern. In cases like this one, the poor results on this test is a known issue caused by the Illumina library prep process. Troubleshoot online if you see poor patterns like in this figure.

Principal Component Analysis (PCA)

One of the most important diagnostic plots is called a principal component analysis (PCA) plot ([Figure 3.15](#)). **A PCA plot is used to visualize how similar data groups are.** In order to do this, an algorithm determines the two variables with the most variation across treatment groups and plots them on a 2D graph. Ideal treatment groups will have each treatment group isolated from one another. PCA plots that are mixed are less likely to produce significant results.

Figure 3.15: PCA Plot (Ideal Example)

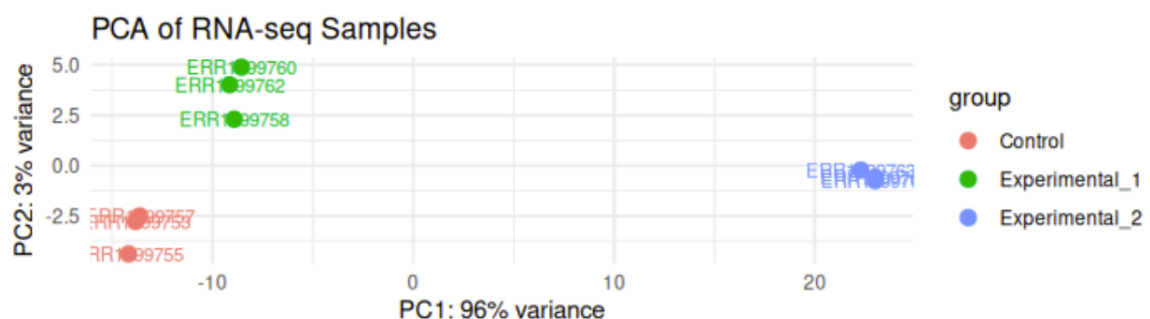


Figure 3.15. An ideal Example of a PCA plot that was generated by our pipeline. This plot shows that our two experimental groups (green and blue data points) are i) spatially isolated from one another (colors are separated from each other), and ii) clumped together within their own experimental group (same colored points are close together). A PCA plot that meets these two criteria makes it likely that our experimental variables drove the variation we observe.

Figure 3.16: Poor PCA Example - Random Clustering

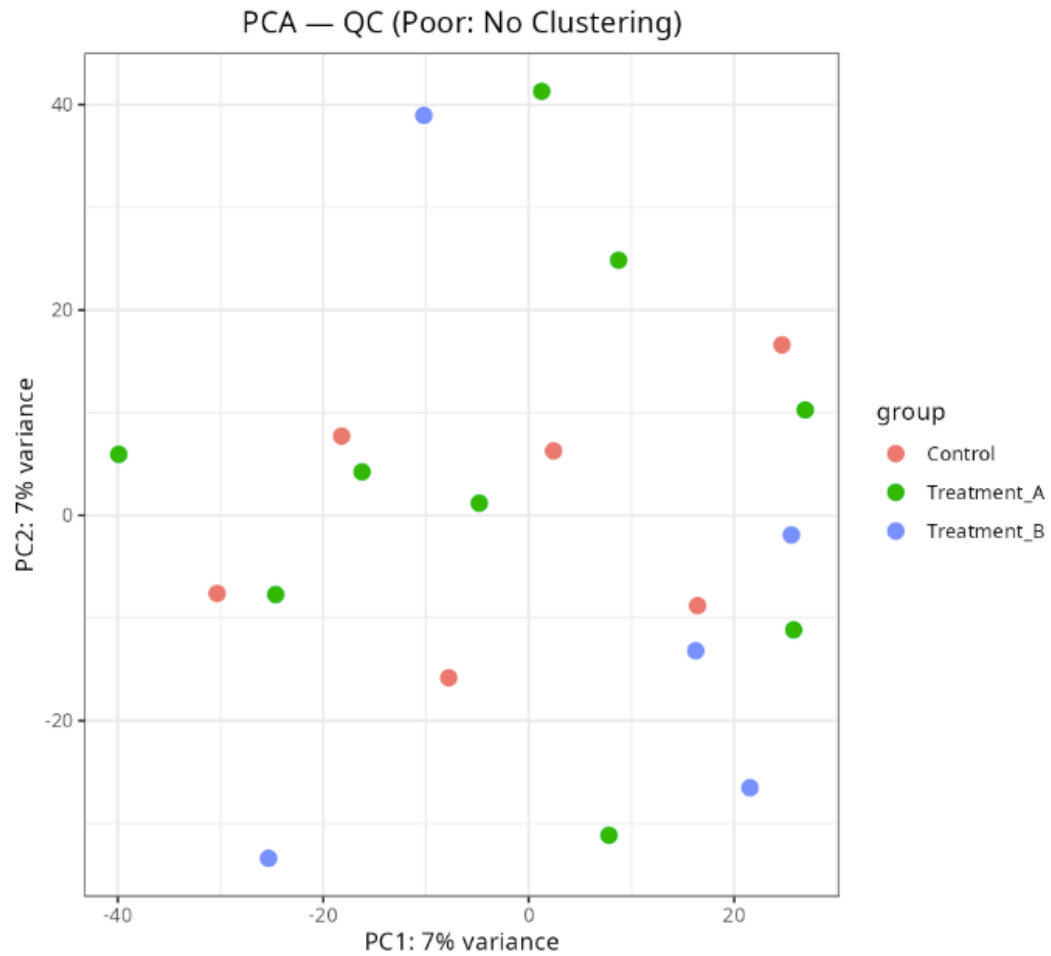


Figure 3.16. An example of a poor PCA plot that doesn't cluster based off of experimental group. If a PCA graph looks like this it is unlikely that the experimental variables that you measured contribute to the variation that you see between the samples. Unfortunately, there is no computational fix for this.

Figure 3.17: Poor PCA Example - Horseshoe Shape

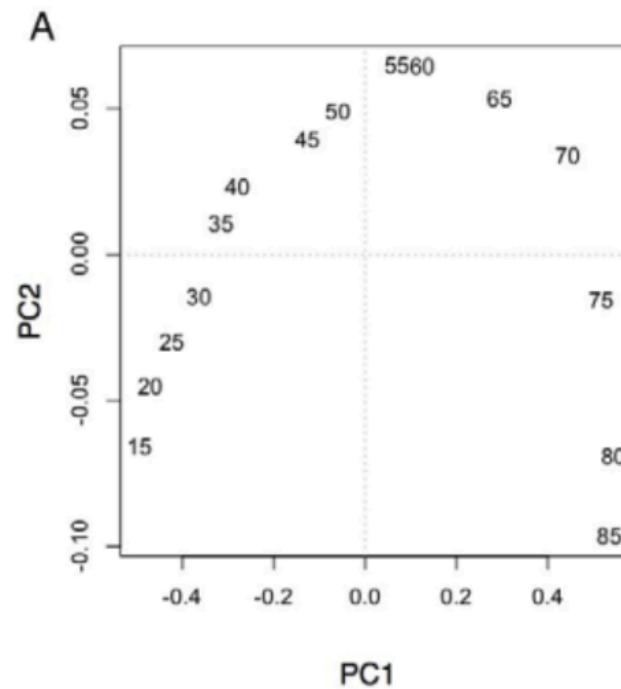


Figure 3.17. An example of a poor PCA plot making a horseshoe shape. Horseshoe shapes and other patterning are often an indicator of a nonlinear pattern. If it is a nonlinear correlation, you can use a program like non-metric multidimensional scaling (NMDS) to account for this.

Figure 3.18: Poor PCA Example - Possible Batch Effect.

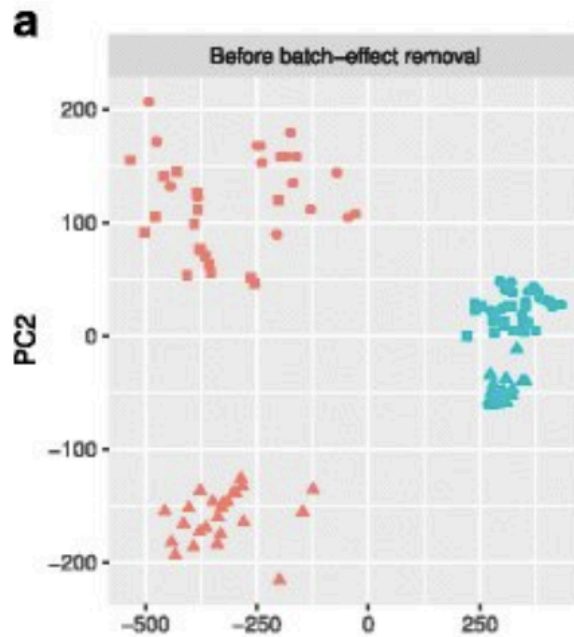


Figure 3.18. An example of a poor PCA example that has signs of batch effect bias. Batch effect bias is a type of bias that comes about from some kind of methodological difference that splits a single experimental group into two distinct groups (Two different machines, protocol done by 2 different people, etc.). The primary sign of a batch effect in this sample is the split between two groups in the red experimental group. There are specialized software that can account for batch effects like this.

Dispersion Plot

Another useful diagnostic plot is the dispersion plot, which measures the amount of reads per gene against that gene's variability. In RNAseq experiments, the amount of biological replicates we have is low (we only have 3 samples per treatment group) To fix this, this dispersion plot shares gene information across all of our samples to account for this limitation.

In this graph, each black dot represents a gene's dispersion (or variability) measured against its own dataset, which contributes to the red “fitted” trendline. If a datapoint's “black dot” is far from the fitted trendline but doesn't hold much weight, gene information from the other datasets is taken into account, and the point is colored blue. Black dots surrounded by blue circles are outliers that were far from the trendline, but held enough weight without considering other datasets.

Figure 3.19: Dispersion Plot Example

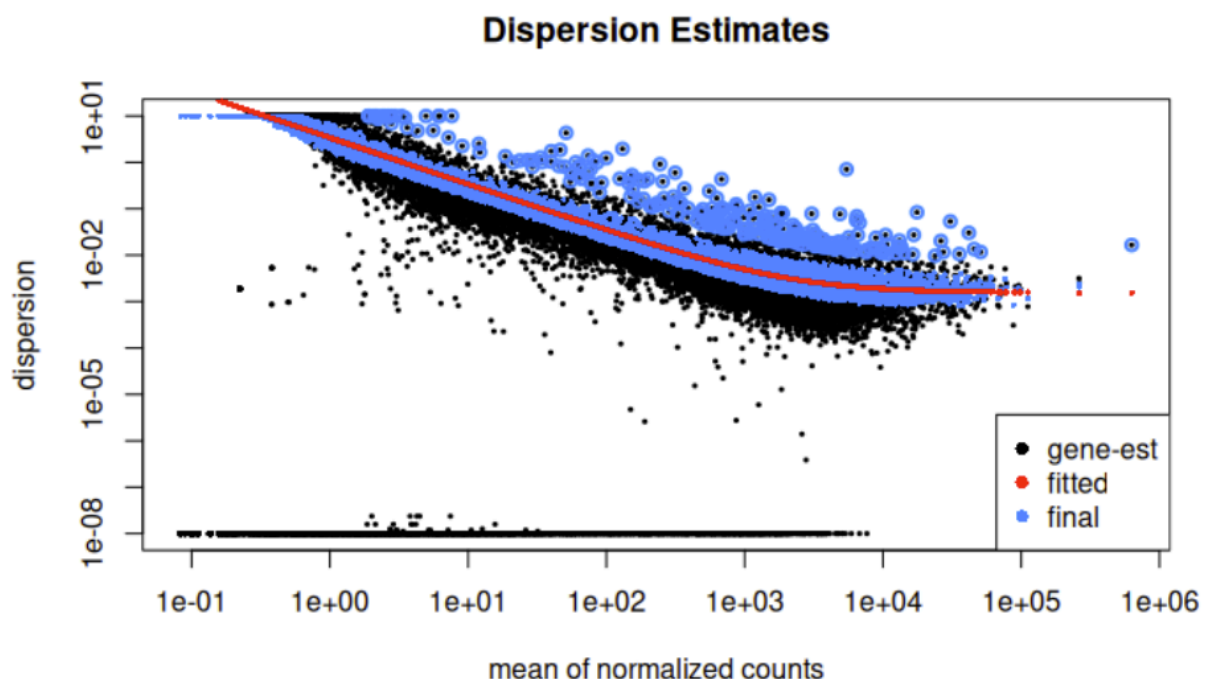


Figure 3.20. An example of an ideal dispersion estimate graph. This plot shows the relationship between the mean normalized counts and dispersion. The black dots represent raw dispersion estimates, the blue dots represent normalized values, and the red line represents the overall trend. You ideally want to see gene estimates follow a general downward trend that dips below 1e-02.

Figure 3.20: Poor Dispersion Estimate Example - Low Shrinkage

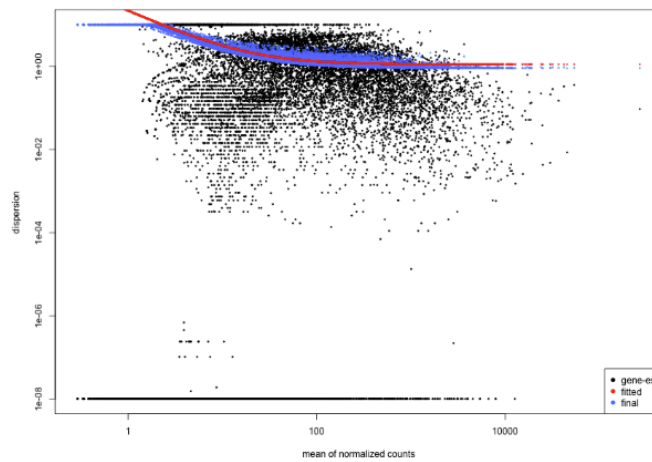


Figure 3.20. An example of a poor dispersion estimate graph due to low shrinkage. This means that the black dots are unusually variable and the software identifies them as closer to outliers. A low shrinkage dispersion plot like this could indicate that your model is not fitting well. Low shrinkage estimates could also be a result of using modified counts instead of raw counts.

Figure 3.21: Poor Dispersion Estimate Example - Irregular Shape

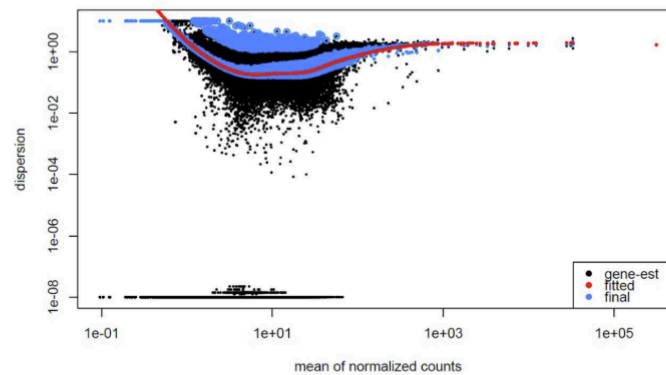


Figure 3.21. A poor example of a dispersion estimate plot that shows an irregular trendline. An irregular trendline and a dense clump of raw gene estimates disconnected from the fitted gene estimates typically indicates an extreme outlier is skewing the data. Oftentimes, data like this can be corrected using an outlier removal program.

Analysis and Graphing

Differentially Expressed Genes Table

Now that we know we can trust our data, we can look into what our data is telling us. The first thing we need to do is determine what genes are over- and underexpressed. A table with the raw differential expression information will be output in a differentially expressed genes “.csv” file located in the results folder. A preview of what this table looks like can be seen in [Figure 3.23](#), and each aspect is explained below.

Figure 3.22: Differentially Expressed Genes Table

```
log2 fold change (MLE): Condition Experimental_1 vs Control
Wald test p-value: Condition Experimental 1 vs Control
DataFrame with 6 rows and 6 columns
```

	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ITGA5	30569.80	2.07277	0.0536958	38.6021	0.00000e+00	0.00000e+00
ADGRL4	1061.53	2.67726	0.0949754	28.1889	7.99376e-175	6.05967e-171
MMP13	6428.28	2.68992	0.0970860	27.7065	5.82275e-169	2.94262e-165
GPNCB	4698.65	2.37570	0.0952698	24.9366	2.98705e-137	1.13217e-133
DHRS2	6133.39	-1.43031	0.0592212	-24.1520	7.10944e-129	2.15573e-125
CHST3	2280.82	1.50797	0.0660750	22.8221	2.76744e-115	6.99287e-112

Figure 3.22. A section of the output table of differentially expressed genes. This will be formatted as a .csv table in our pipeline.

Each row represents a gene with a `padj` value less than .05. There are two important values that we want to pay attention to in this table: the `log2FoldChange` and the `padj`. The `log2FoldChange` represents how up or downregulated that specific gene is. The more up/downregulated a gene is, the more likely that dysregulation is to affect the overall system. This means that big numbers (positive or negative) are especially interesting.

`padj` values represent a p-value that has been manipulated to be more accurate. As a reminder, the p-value is the probability that our result happened by chance alone. In science, it is standard to treat any p-value below .05 (meaning that there is less than a 1/20 chance that we accidentally found meaning in a meaningless result) as significant – or trustworthy. However, when we are looking at thousands of samples like we are doing here, this means that even if we look at a sample with a

p-value of .05 there could be hundreds of false positives. A adjusted p-value accounts for this in a couple different ways. The most popular of which is multiplying the p-value by the number of samples you have. Additionally while any p-value under .05 is considered significant, the lower the p-value, the more confident you can be that the result is significant.

Volcano Plot - Visualize Significant Genes

Another useful plot for RNA differential expression is a volcano plot. Named this way because it looks like a volcano, this plot visually demonstrates how many genes are up/downregulated. The X-axis of this graph is the Log_2 of the “fold change” or how big the change is. The Y-axis is the $-\log_{10}$ of the P-value or how confident we are in that value. What we want to look for are the points in the top corners that are both high up/down regulation and a high confidence. To plot a volcano graph use the following code.

Figure 3.23: Ideal Example of a Volcano Plot

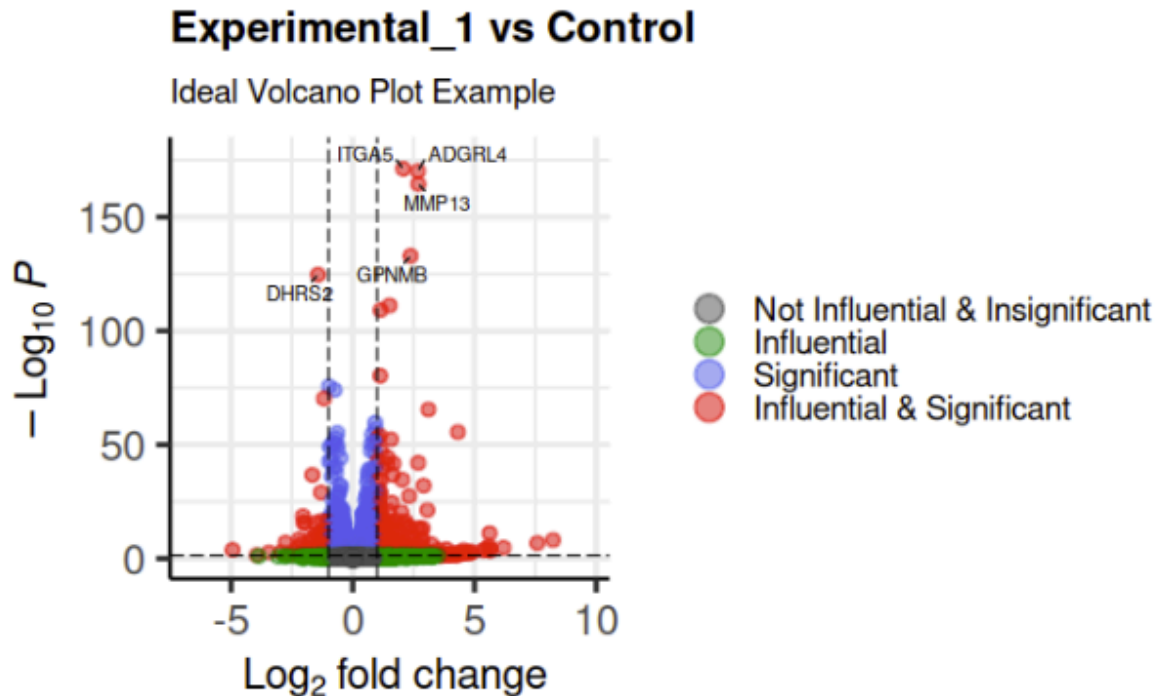


Figure 3.23. An ideal example of a volcano plot that was outputted by the RNAseq Pipeline. Each dot represents a gene plotted on a $\log_2$ fold change of significance on the X-axis and a $-\log_{10}$ of the P-Value on the Y-axis. This means the closer to the left or right the point is the more influential the point is and the higher the point, the more confident we are in that gene. Influential points are noted by a $\log_2$ fold change or greater and significance was determined with a P value of .05. Genes are color coordinated on based on influence and significance. Red genes that are labeled both significant and influential are the most important points to pay attention to and the 5 most significant genes are labeled.

Figure 3.24: Poor Volcano Plot - Significant Not Influential

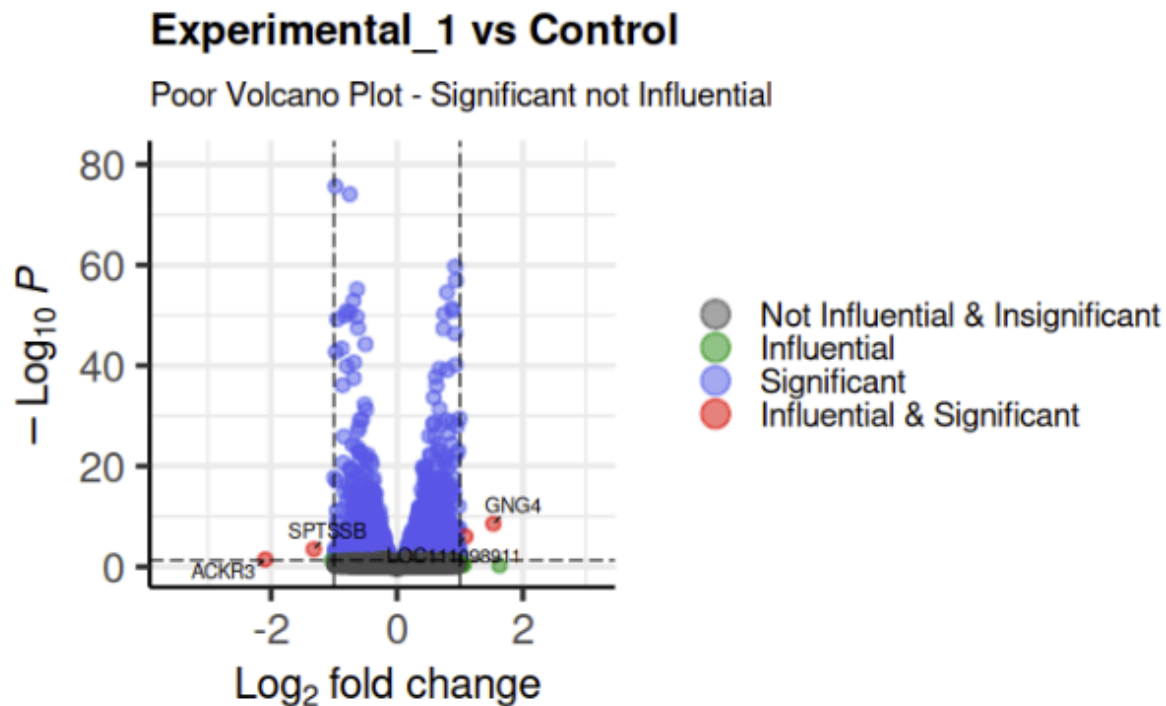


Figure 3.24. An example of a volcano plot that has many significant but not influential genes. Genes are represented as dots that are color coordinated based on if it's influential ($> 1 \log_2$ fold dysregulation) and significant ($P < 0.05$). While there are many significant genes, most don't have an influential dysregulation in gene expression. This means that while we can be confident that many genes are either up- or down-regulated when compared to the control group, most of those genes are not dysregulated enough to be interesting. However, there are a couple of genes that are both influential and significant (red). These points are interesting and should be looked at further.

Figure 3.25: Poor Volcano Plot - Influential Not Significant

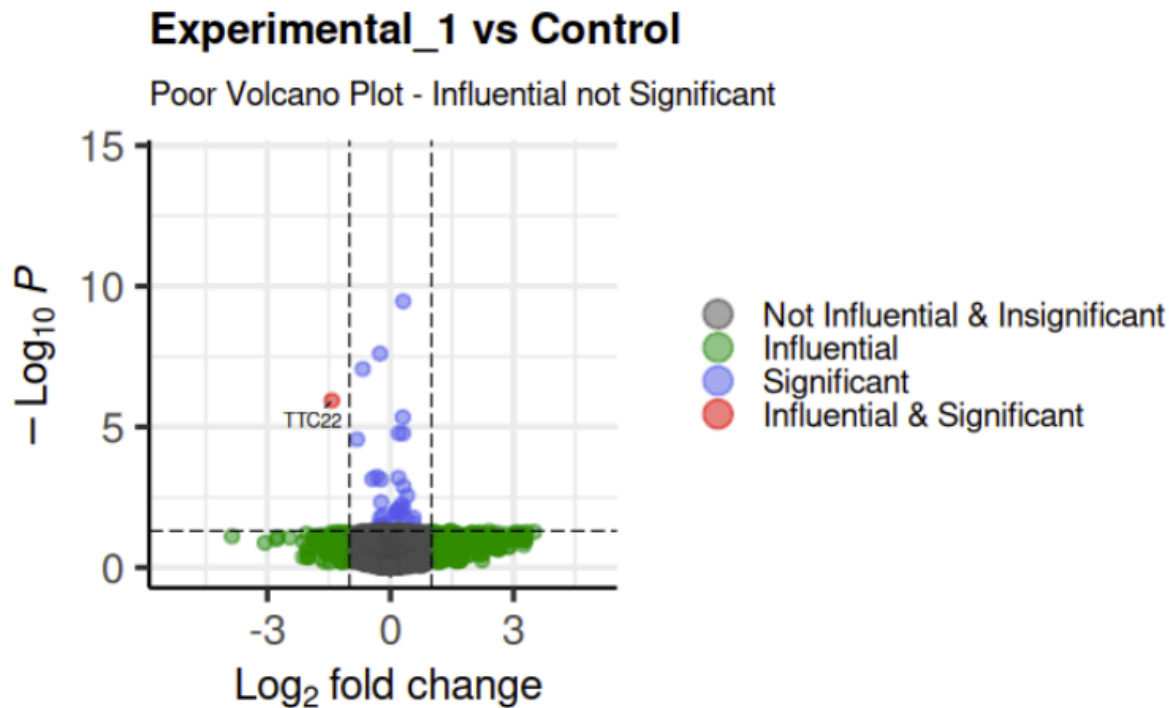


Figure 3.25. An example of a volcano plot with many influential, but not significant, genes. Genes are plotted and color coordinated based on if it is influential ($> \text{Log}_2$ fold change in gene expression) and/or significant ($P < 0.05$). While there are many influential genes, very few of those genes are significant enough to draw meaningful conclusions from. There is one exception labeled in red (gene *TTC22*) that is both influential and significant and should be looked into more. Influential, but not significant points (green) likely need an experiment with more statistical significance before drawing meaningful conclusions.

GSEA Plot - Contextualize in Biological Pathways

One important figure that is used to contextualize our results in real biological pathways GSEA plot seen in [Figure 3.27](#). This plot ranks your genes from most overexpressed to most underexpressed across the X-axis. The plot then looks at a specific biological pathway, in this case, the epithelial-mesenchymal transition pathway (seen in the title). Once the pathway is specified, every gene that is related to that pathway is marked on the bottom of the graph (this is the barcode pattern just above the X-Axis). The green line then scans across the X-axis, starting at 0 and rising each time it hits one of the barcoded genes, and falling each time it hits a gene that isn't barcoded.

A green line that rises sharply at the beginning at the beginning and/or end of the graph are particularly of interest. This would mean that Many of the most upregulated (beginning) and/or downregulated (end) genes are associated with our pathway of interest. A line that stays relatively steady or doesn't make a defined "sideways S-like" curve is particularly suspect. Additionally, there are thousands of different pathways in an organism, and it would take years to manually check every pathway. To speed this up a bit, there are R-packages with algorithms that can find the most influential biological pathways in seconds. The implementation of our code can be seen in [Chapter 2: Making a GSEA Plot](#).

Figure 3.26: Ideal GSEA plot

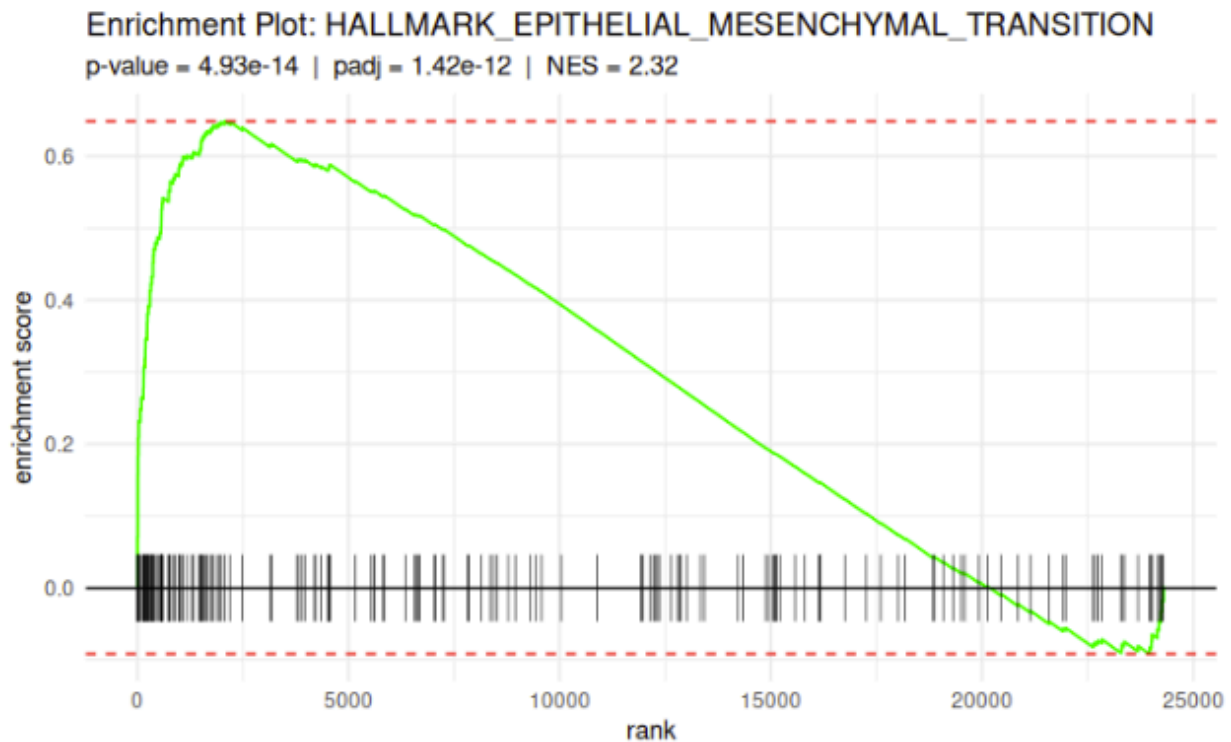


Figure 3.26. An ideal GSEA plot generated by our pipeline. This plot ranks each gene from the most overexpressed to the most underexpressed across the X-axis. The plot then looks at a specific metabolic pathway and marks each gene associated with that pathway as a black line where that gene was ranked. The green enrichment line then travels across the ranked gene list raising when reaching a gene associated with the pathway and lowering when it doesn't. An ideal curve like this will increase sharply at both the low and high-ranked genes, indicating many over- and underexpressed genes associated with the pathway.

Chapter 4 - Conclusion

Hopefully, this thesis has given you the materials needed to kickstart your bioinformatics journey here at ECU. Chapter 1 described what bioinformatics is, what skills a bioinformatician needs, and some reference material on skills needed to set yourself up for success in your bioinformatics journey. Chapter 2 went into detail on the function of, as well as the thought process of writing, a bioinformatic pipeline I wrote for RNAseq differential expression analysis. Finally, Chapter 3 explained how to read the complex QC and analysis figures produced by this pipeline, with visual examples of what these graphs look like when things go wrong and why they occur.

I understand that the information in this thesis may feel like you're drinking from a firehose. While I attempted to break down these concepts and explanations as best as I could while maintaining accuracy, much of this information may be overwhelming. Rest assured, this is expected and completely normal to not fully understand everything the first time through. This thesis was not meant to be a "one and done" read, but more of a guide to be referenced as needed. Therefore, as you reach times in your research when you need to make or read a [volcano plot](#), [GSEA plot](#), or [Python pipeline](#), I highly recommend you refer back to this thesis throughout your journey as a researcher.

As for immediate future directions of this thesis, the design of the pipeline itself (down to the model organism used) is meant to fit into the larger ECU Bulldog Fetal Anasarca Project being led by Dr. Alicia Howard. While unforeseen setbacks have kept our current data from being usable, the pipeline should be directly usable with data from

at least two biological replicates of “water puppies” and “healthy puppies” (however, I highly recommend more). Currently, commercial differential analysis services cost around \$600 a sample⁵¹. While samples will still need to be sequenced out of house, I have gotten this pipeline to run on an 8-core processor for the cost of the electricity it uses, and it can be modified to work with future techniques. The full pipeline, along with a copy of this thesis, can be found at my GitHub [here](#)

As for the longer-term future directions of this thesis, I hope that this becomes the start of a “living document” to be added to by future students. While this thesis was long, it only covered NGS RNAseq. Bioinformatic techniques evolve alongside our understanding of biology and will likely be expedited with AI advancement. If future students want to add to or modify a copy of this document to reflect technological advancements, add techniques, and help future students, I highly encourage it.

Thank you for taking the time to read this thesis. If you are an ECU student who has any questions or concerns, feel free to contact me at caden.elizalde@gmail.com. Good luck in your journey into bioinformatics.

Glossary

A

Argument - something that is added to the command that runs a BASH command, which gives a piece of information that affects how the script runs.

B

Batch Processing - A group (or batch) of inputs that are fed (or processed) through a specific section of code one by one. Therefore, being processed identically.

BIOS (Basic Input/Output System) - Firmware that checks to see that your computer is good to go before loading your OS.

C

Channels - In conda, a channel represents a website that stores and hosts conda packages.

Command Line - The text based interface between you and the computer.

Command Line Interface (CLI) - The same as the Command Line

Conda - A package management software.

CPU - "Central Processing Unit" the part of the computer that deals with heavy calculations.

CRAN - Stands for "Comprehensive R Archive Network" and is a repository that stores and maintains R packages and language versions.

D

Debian - One of the 7 base distributions that has gained popularity for general use.

Distribution/Distro - A specialized version of Linux designed for a specific use case. subdivided into 7 base distributions, with many more specialized.

E

EMT - "Epithelial-Mesenchymal Transition"

Cancer cell's transition from epithelial cells to mesenchymal cells. A key step in cancer metastasis.

ENA - "European Nucleotide Archive" one of the many gene repositories where you can download public genomic Datasets.

Environment - an isolated directory that contains a specific collection of packages, including a particular Python or R version, libraries, and other software, along with their dependencies.

F

G

Genome - The nucleic acid sequence that can be found in an individual.

GSEA - "Gene Set Enrichment Analysis" A differential expression plot that shows how genes are over and underexpressed in a given metabolic pathway

H

The Human Genome Project - A scientific project that lasted from 1990 - 2003 that sought to map the entirety of the Human Genome.

I

Integrated Development Environment (IDE) - A piece of software that makes it more user friendly to run code.

ISO Image - A single file that's a perfect representation of an entire OS.

J

K

Kernel - The very base level of a computer system.

L

Lint - A coding tool that detects syntax errors in code.

Linux - Short for “Libre Unix” is an open source clone of Linux. Colloquially and functionally identical to Unix.

Linux Mint - An Ubuntu based specialized distribution notable for being user friendly.

M

MacOS - The operating system made by Apple that runs on Macintosh Computers.

N

O

Open Source - Typically free software with code that is open and available to the public.

OS - “Operating System” Protocols that allow people to interact with the internals of the machine.

P

Parameter - Something that is added to the command that runs a Python script, which gives a piece of information that affects how the script runs.

Phred Score - A quality score for nucleotides that measures a “1 in X” chance that the nucleotide is wrong (i.e. Phred Score of 20 = 1/20 chance the nucleotide is wrong).

Plugin - An add on to a software that allows for a specific feature

Q

Quality Score - An integer value representing the estimated probability of an error.

R

Reference Sequence (RefSeq) - A genome that is representative of the typical individual of a species.

Repository (Repo) - A central digital storage location for organizing, managing, and sharing project files, code, and data.

S

Script - A sequence of commands or instructions used to run a series of predefined tasks.

SHA-256 - A cryptographic hash function that acts as a 256-bit string of characters that acts as a digital fingerprint.

shRNA - Short Hairpin RNA

Syntax - The way code is “phrased.”

T

U

Ubuntu - One of 7 base distributions specialized for general use. The base Distribution of Linux Mint.

Unix - Colloquially identical to Linux. A proprietary command line based operating system made by AT&T’s Bell Lab. Unix gained popularity due to its speed, efficiency, and simplicity.

URL - The web address of a unique resource on the internet, such as a web page or file.

V

Variance Stabilizing Transformation - A type of data manipulation useful for allowing data that doesn’t follow a normal distribution, to be analyzed by data that needs to be normally distributed.

Volcano Plot - A differential expression plot that visually displays genes based on influence and significance.

W

Wildcard - A character (typically “*”) that symbolizes a placeholder for any number of characters.

X

Y

Z

Appendix A - Important Commands

Command	Brief Description	Notable options	Usage Example
apt	A software package management tool	<code>install</code> : Install a software package <code>upgrade</code> : update all installed software <code>remove</code> : Remove a software package	<code>apt install <software></code>
cat	Read or combine files.	<code>-n</code> : number output lines	<code>cat <file></code>
cd	Navigate to a specified directory		<code>cd <directory></code>
chmod	Change file permissions.	<code>-v</code> : Have the terminal output a result for everything it looks at <code>-f</code> : Silence most error messages <code>-c</code> : Only outputs a terminal message when a change is made	<code>chmod <file></code>
cp	Copy a file.	<code>-f</code> : Force the command to go through even if an error message occurs <code>-i</code> : Pop up an interactive prompt before an overwrite.	<code>cp <source file> <destination file></code>
echo	Repeat a string of text back to you.		<code>echo "<string to repeat>"</code>
grep	Search a file or location for a pattern of text		<code>grep <pattern> <file></code>
rm	Remove a file or directory.	<code>-r</code> : "recursive," do the same to files in all subdirectories <code>-f</code> : Force remove the file	<code>rm <file></code>
sudo	Grant administrative access to a command.		<code>sudo <command requiring admin access></code>
history	Gets a numbered list of previously run commands	<code><number></code> : Immediately following the command with a number will only list that many commands back <code>-w</code> : Writes the history to a history file	<code>history <number> <options></code>
less	Shows large files one page at a time	<code>-c</code> : Clears screen before displaying each page	<code>less <file></code>
ls	List files and directory in the current.	<code>-l</code> : List permissions of the files <code>-a</code> : Lists hidden files and directory (starting with <code>.</code>) as well	<code>ls <options></code>

man	Look at the manual for a given command.	-f: Displays a short description if available.	man <options> <command>
mkdir	Make a directory to the current working directory	-p: Make parent directories if needed	mkdir <options> <directory>
more	Read a file		more <file>
mv	Rename or move a file.	-f: Don't prompt before overwrite -i: Prompt the user before an overwrite occurs	Rename: mv <old name> <new name> Move: mv <source directory> <destination directory>
nano	Access an in terminal text editor for a given file.		nano <file>
ps	Shows currently running processes	-A: Shows all processes. -f: Full format listing.	ps
pwd	Shows the current working directory.		pwd
tail	Shows the end of a file	-n <number>: specifies a number of lines back to show. -f: Continuously output new lines being created	tail <file>
tee	Takes a standard input and writes it to the standard output as well as another specified file.	-a: append to file rather than overwrite -i: ignore interrupts	tee <file>
touch	Create a file		touch <new filename>
watch	Repeats a command over time and prints the output	-n: how many seconds pass before the next command repeat	watch <command>
wget	Retrieves a file from the internet.	-v: "Verbose" displays all available output. -q: "quiet" suppresses all output besides for error messages.	wget <URL>

Appendix B - BASH Aliasises & Shortcuts

Shortcut	What it Does	Example
!!	Rerun last command	<pre>> echo "hello world" hello world > !! hello world</pre>
!\$	Repeat last argument	<pre>> mkdir folder > cd !\$ (you are now in the folder directory)</pre>
!<number>	Repeat the <number> command in the terminals history (useful when paired with the <code>history</code> command)	<pre>> history 3 1 ... 2 echo "hello world" 3 ... > !2 hello world</pre>
!-<number>	Repeat the command <number> steps back	<pre>> echo "hello" hello > echo "world" world > !-2 hello</pre>
*	Wildcard: matches to anything	<pre>> ls *.txt file1.txt file2.txt File2.txt (shows only files ending in .txt)</pre>
\$USER	Home Directory of the current user (useful if you want a script that works on any computer regardless of user)	<pre>> pwd \$USER /home/caden (Will show the current user instead of Caden)</pre>
~	Shortcut for home directory	<pre>> cd ~ > pwd /home/<user> (calls the home directory)</pre>
>	Overwrite to file	<pre>> cat sample.txt hello world > "my name is Caden" > sample.txt</pre>

		<pre>> cat sample.txt my name is Caden</pre>
>>	Append to file	<pre>> cat sample.txt hello world > "my name is Caden" >> sample.txt > cat sample.txt hello world my name is Caden</pre>
" "	Strings in double quotes except certain special characters and variables	<pre>> echo "I am \$USER" I am caden</pre>
' '	Strings in single quotes will be treated as literal	<pre>> echo 'I am \$USER' I am \$USER</pre>
;	Will immediately run the next command when placed between 2 commands	<pre>> echo "hello" ; echo "hello" Error: ... hello</pre>
&&	Putting && between two commands will only run the second command if the first one succeeds	<pre>> ech "hello" ; echo "hello" Error: ...</pre>

Appendix C - Useful Bioinformatics Software

Software	Description
Bioconductor	A project that contains useful R packages for genomic data analysis.
Conda	A package management software used to ensure software compatibility.
FastQC	Used to create documentation on the quality of reads.
featureCounts	Creates a summary table of reads mapped to genes.
HISAT2	Used for alignment mapping for NGS sequencing data.
HISAT2-build	Used to build index files from a refseq.
MultiQC	Used to combine the documentation of FastQC into an easy to read document.
pandas	Standard Python package for data manipulation
Python	A simple object based coding language that is useful for scripting.
R	A coding language optimized for statistical analysis.
R Studio	An IDE used to run R code.
rpy2	A software that acts as a bridge between R and Python

Sources

1. Albert, I. (2021). *The Biostar Handbook: 2nd Edition* (2nd ed.).
2. Can, T. (2014). Introduction to Bioinformatics. In M. Yousef & J. Allmer (Eds.), *miRNomics: MicroRNA Biology and Computational Analysis* (pp. 51–71). Humana Press.
https://doi.org/10.1007/978-1-62703-748-8_4
3. Chen, J., Wang, X., Ma, A., Wang, Q.-E., Liu, B., Li, L., Xu, D., & Ma, Q. (2022). Deep transfer learning of cancer drug responses by integrating bulk and single-cell RNA-seq data. *Nature Communications*, 13(1), 6494. <https://doi.org/10.1038/s41467-022-34277-7>
4. Cho, C.-S., Xi, J., Si, Y., Park, S.-R., Hsu, J.-E., Kim, M., Jun, G., Kang, H. M., & Lee, J. H. (2021). Microscopic examination of spatial transcriptome using Seq-Scope. *Cell*, 184(13), 3559–3572.e22. <https://doi.org/10.1016/j.cell.2021.05.010>
5. Ding, J., Sharon, N., & Bar-Joseph, Z. (2022). Temporal modelling using single-cell transcriptomics. *Nature Reviews Genetics*, 23(6), 355–368.
<https://doi.org/10.1038/s41576-021-00444-7>
6. Greener, J. G., Kandathil, S. M., Moffat, L., & Jones, D. T. (2022). A guide to machine learning for biologists. *Nature Reviews Molecular Cell Biology*, 23(1), 40–55.
<https://doi.org/10.1038/s41580-021-00407-0>
7. Harshil Patel, Jonathan Manning, Phil Ewels, Maxime U Garcia, Alexander Peltzer, Rickard Hammarén, Olga Botvinnik, Adam Talbot, Friederike Hanssen, Gregor Sturm, nf-core bot, Matthias Zepper, Denis Moreno, Pranathi Vemuri, Mahesh Binzer-Panchal, Ezra Greenberg, silviamorins, Lorena Pantano, Robert Syme, ... Chris Cheshire. (2025). *nf-core/rnaseq: Nf-core/rnaseq v3.22.1 - Polished Palladium Penguin* (Version 3.22.1) [Computer software]. Zenodo. <https://doi.org/10.5281/ZENODO.17833254>
8. *Homo sapiens* (ID 214799)—BioProject—NCBI. (n.d.). Retrieved December 5, 2025, from <https://www.ncbi.nlm.nih.gov/bioproject/?term=PRJNA214799>
9. *Human Data*. (n.d.). Retrieved December 5, 2025, from <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE49712>

10. Jaganathan, K., Kyriazopoulou Panagiotopoulou, S., McRae, J. F., Darbandi, S. F., Knowles, D., Li, Y. I., Kosmicki, J. A., Arbelaez, J., Cui, W., Schwartz, G. B., Chow, E. D., Kanterakis, E., Gao, H., Kia, A., Batzoglu, S., Sanders, S. J., & Farh, K. K.-H. (2019). Predicting Splicing from Primary Sequence with Deep Learning. *Cell*, 176(3), 535-548.e24.
<https://doi.org/10.1016/j.cell.2018.12.015>
11. Li, X., & Wang, C.-Y. (2021). From bulk, single-cell to spatial RNA sequencing. *International Journal of Oral Science*, 13(1), 36. <https://doi.org/10.1038/s41368-021-00146-0>
12. Liao, J., Qian, J., Fang, Y., Chen, Z., Zhuang, X., Zhang, N., Shao, X., Hu, Y., Yang, P., Cheng, J., Hu, Y., Yu, L., Yang, H., Zhang, J., Lu, X., Shao, L., Wu, D., Gao, Y., Chen, H., & Fan, X. (2022). De novo analysis of bulk RNA-seq data at spatially resolved single-cell resolution. *Nature Communications*, 13(1), 6498. <https://doi.org/10.1038/s41467-022-34271-z>
13. Mao, D. (2025). The use of artificial intelligence in RNA sequencing: A review. *Advances in Engineering Innovation*, 16(5), 90–94. <https://doi.org/10.54254/2977-3903/2025.23332>
14. Mortazavi, A., Williams, B. A., McCue, K., Schaeffer, L., & Wold, B. (2008). Mapping and quantifying mammalian transcriptomes by RNA-Seq. *Nature Methods*, 5(7), 621–628.
<https://doi.org/10.1038/nmeth.1226>
15. Ozsolak, F., & Milos, P. M. (2011). RNA sequencing: Advances, challenges and opportunities. *Nature Reviews Genetics*, 12(2), 87–98. <https://doi.org/10.1038/nrg2934>
16. Perelo, L. W., Gabernet, G., Straub, D., & Nahnsen, S. (2024). How tool combinations in different pipeline versions affect the outcome in RNA-seq analysis. *NAR Genomics and Bioinformatics*, 6(1), lqae020. <https://doi.org/10.1093/nargab/lqae020>
17. *Transcriptomic responses of Arabidopsis thaliana l... (ID 674629)—BioProject—NCBI*. (n.d.). Retrieved December 5, 2025, from <https://www.ncbi.nlm.nih.gov/bioproject/?term=PRJNA674629>
18. VanInsberghe, M., Van Den Berg, J., Andersson-Rolf, A., Clevers, H., & Van Oudenaarden, A. (2021). Single-cell Ribo-seq reveals cell cycle-dependent translational pausing. *Nature*, 597(7877), 561–565. <https://doi.org/10.1038/s41586-021-03887-4>
19. Wang, Y., Ma, X., Xu, E., Huang, Z., Yang, C., Zhu, K., Dong, Y., & Zhang, C. (2024). Identifying squalene epoxidase as a metabolic vulnerability in high-risk osteosarcoma using an artificial

intelligence-derived prognostic index. *Clinical and Translational Medicine*, 14(2), e1586.

<https://doi.org/10.1002/ctm2.1586>

20. Zaitsev, A., Chelushkin, M., Dyikanov, D., Cheremushkin, I., Shpak, B., Nomie, K., Zyrin, V., Nuzhdina, E., Lozinsky, Y., Zotova, A., Degryse, S., Kotlov, N., Baisangurov, A., Shatsky, V., Afenteva, D., Kuznetsov, A., Paul, S. R., Davies, D. L., Reeves, P. M., ... Bagaev, A. (2022). Precise reconstruction of the TME using bulk RNA-seq and a machine learning algorithm trained on artificial transcriptomes. *Cancer Cell*, 40(8), 879-894.e16.
<https://doi.org/10.1016/j.ccell.2022.07.006>
21. Chang, J. (2015). Core services: Reward bioinformaticians. *Nature*, 520(7546), 151–152.
<https://doi.org/10.1038/520151a>
22. *Overview of the GNU System—GNU Project—Free Software Foundation*. (n.d.). Retrieved January 4, 2026, from <https://www.gnu.org/gnu/gnu-history.en.html>
23. Ritchie, D. M., & Thompson, K. (1978). The UNIX Time-Sharing System†. *Bell System Technical Journal*, 57(6), 1905–1929. <https://doi.org/10.1002/j.1538-7305.1978.tb02136.x>
24. College, S. M.-W. (n.d.). *What Is an ISO File?* Lifewire. Retrieved January 5, 2026, from <https://www.lifewire.com/iso-file-2625923>
25. *Linux Mint Installation Guide*. (2024). Linux Mint.
https://linuxmint-installation-guide.readthedocs.io/_/downloads/en/latest/pdf/
26. *Etcher | Etcher*. (n.d.). Retrieved January 8, 2026, from <https://etcher.pages.dev/>
27. *Conda Documentation—Conda 25.11.1 documentation*. (n.d.). Retrieved January 22, 2026, from <https://docs.conda.io/projects/conda/en/stable/index.html>
28. EMBL-EBI. (2026). *ENA Browser*. Ebi.ac.uk. <https://www.ebi.ac.uk/ena/browser/view/ERP022283>
29. Team, G. (2026). *Experiment < Expression Atlas < EMBL-EBI*. Ebi.ac.uk.
<https://www.ebi.ac.uk/gxa/experiments/E-MTAB-5628/Results>
30. *NCBI Genome Data Viewer*. (2026). Nih.gov. <https://www.ncbi.nlm.nih.gov/gdv/>
31. *UCSC Genome Browser Home*. (2026). Ucsd.edu. <https://genome.ucsc.edu/>
32. *Ensembl Genomes*. (2026). Ensemblgenomes.org. <https://ensemblgenomes.org/>
33. *FlyBase*. (2026). *FlyBase Homepage*. Flybase.org. <https://flybase.org/>

34. HISAT2. (2021). HISAT2. <https://daehwankimlab.github.io/hisat2/>
35. Research & analysis changes in U.S. family finances from 2019 to 2022. (n.d.).
<https://www.federalreserve.gov/publications/files/scf23.pdf>
36. Bakker, J. D. (2026, January 2). PCA. *Applied Multivariate Statistics in R*; University of Washington. <https://uw.pressbooks.pub/appliedmultivariatestatistics/chapter/pca/>
37. Lewis, P. D., & Menzies, G. E. (2015). Vibrational spectra, principal components analysis and the horseshoe effect. *Vibrational Spectroscopy*, 81, 62–67.
<https://doi.org/10.1016/j.vibspec.2015.10.002>
38. Kennedy, W. B. (2024, November 2). A Simple Example Using PCA for Outlier Detection. Medium; TDS Archive.
<https://medium.com/data-science/a-simple-example-using-pca-for-outlier-detection-ab2773b98e4a>
39. Zhu, X., Wolfgruber, T. K., Tasato, A., Arisdakessian, C., Garmire, D. G., & Garmire, L. X. (2017). Granatum: a graphical single-cell RNA-Seq analysis pipeline for genomics scientists. *Genome Medicine*, 9(1). <https://doi.org/10.1186/s13073-017-0492-3>
40. Piper, M. (2017, May 12). Gene-level differential expression analysis with DESeq2. Introduction to DGE - ARCHIVED.
https://hbctraining.github.io/DGE_workshop/lessons/04_DGE_DESeq2_analysis.html
41. The ENCODE Consortium. (2011, June). Standards, guidelines and best practices for RNA-seq (V1.0).
https://genome.ucsc.edu/ENCODE/protocols/dataStandards/ENCODE_RNAseq_Standards_V1.0.pdf
42. Biostar Forum, Featurecounts: Extremely low rate of “successfully assigned alignments.” (2020).
<https://www.biostars.org/p/448496/>
43. MultiQC. (2016). MultiQC Report. Cancer.gov.
https://bioinformatics.ccr.cancer.gov/docs/b4b/data/module2_outputs/multiqc_report_snidget.html
44. cageseq: Output. (2018). Nf-Co.re. <https://nf-co.re/cageseq/output>

45. *Biostar Forum, fastqc result: whether trim off low quality reads or not* (2020).
<https://www.biostars.org/p/448709/>
46. 11. *Advanced quality control with MultiQC copy - Bioinformatics for Beginners 2022*. (2022).
Cancer.gov.
<https://bioinformatics.ccr.cancer.gov/docs/b4b/11.-Advanced-quality-control-with-MultiQC%20copy/>
47. Wu, H., Zhang, Z., Hu, S., & Yu, J. (2012). *On the molecular mechanism of GC content variation among eubacterial genomes*. *Biology Direct*, 7(1). <https://doi.org/10.1186/1745-6150-7-2>
48. *Reddit - The heart of the internet*. (2025). *Reddit.com*.
https://www.reddit.com/r/bioinformatics/comments/1imuyho/fastqc_gc_content/
49. *fastqc gc-content peak and adapter content* (2020). <https://www.biostars.org/p/258230/>
50. Baccarella, A., Williams, C. R., Parrish, J. Z., & Kim, C. C. (2018). *Empirical assessment of the impact of sample number and read depth on RNA-Seq analysis workflow performance*. *BMC Bioinformatics*, 19(1), 423. <https://doi.org/10.1186/s12859-018-2445-2>
51. *Service Fees | Nebraska Center for Biotechnology | Nebraska*. (n.d.). Retrieved April 22, 2026, from <https://biotech.unl.edu/bioinformatics/service-fees/>